

Part I — Dataset Preparation and Feature Engineering

Composer Classification Using Deep Learning

Introduction

Music is one of the most expressive forms of art, with each composer developing a distinctive style through melody, harmony, rhythm, tempo, and musical structure. Identifying the composer of an unfamiliar piece can be challenging, particularly for listeners without formal musical training. Advances in deep learning provide an opportunity to automatically learn these musical patterns and classify compositions based on their underlying characteristics.

The objective of this project is to develop a deep learning system capable of identifying the composer of a musical piece using its MIDI representation. The project focuses on compositions by Bach, Beethoven, Chopin, and Mozart and applies two deep learning architectures: Long Short-Term Memory (LSTM) networks and Convolutional Neural Networks (CNNs). These models will be used to recognize distinctive musical patterns and classify each composition according to its composer.

To accomplish this objective, the project follows a complete machine learning workflow that includes data collection, data preprocessing, data augmentation, feature extraction, model development, model training, evaluation, and optimization. Preparing the musical data in a structured and consistent format is essential so that the deep learning models can effectively learn meaningful relationships from the MIDI sequences.

This notebook presents the data preparation portion of the project. It includes dataset selection and exploration, MIDI file validation, train-validation-test splitting, data augmentation, feature extraction, composer label encoding, sequence normalization, and TensorFlow-compatible sequence preparation. The processed data produced in this notebook will be used as input for the LSTM and CNN models developed in the later stages of the project.

Project Methodology

The complete team project follows these steps:

1. Data Collection
2. Data Preprocessing
3. Data Augmentation
4. Feature Extraction
5. Model Building
6. Model Training
7. Model Evaluation
8. Model Optimization

The dataset already contains MIDI files, so no conversion from another musical-score format is required. This notebook selects the four required composers, validates the MIDI files, extracts musical features, and prepares TensorFlow-compatible sequences.

The notebook ends after producing the processed datasets required for model development.

1. Environment Setup

This section installs and imports the libraries required for MIDI processing, numerical analysis, visualization, dataset splitting, and TensorFlow sequence preparation.

```
In [ ]: # Install the libraries required for MIDI processing and data preparation.

# !pip install pretty_midi tensorflow scikit-Learn pandas numpy matplotlib
```

```
In [ ]: # Import standard Python libraries.
from pathlib import Path
from collections import defaultdict
import json
import warnings

# Import data-processing libraries.
import numpy as np
import pandas as pd

# Import the visualization library.
import matplotlib.pyplot as plt

# Import the MIDI-processing library.
import pretty_midi

# Import the dataset-splitting function.
from sklearn.model_selection import train_test_split

# Import TensorFlow and the Keras padding utility.
import tensorflow as tf
from tensorflow.keras.utils import pad_sequences

# Reduce unnecessary warning messages.
warnings.filterwarnings("ignore")

# Set random seeds to support reproducible results.
RANDOM_SEED = 42

np.random.seed(RANDOM_SEED)
tf.random.set_seed(RANDOM_SEED)

print("TensorFlow version:", tf.__version__)
print("NumPy version:", np.__version__)
print("Pandas version:", pd.__version__)
print("Environment setup completed successfully.")
```

TensorFlow version: 2.16.1

NumPy version: 1.24.3

Pandas version: 2.2.3

Environment setup completed successfully.

2. Dataset Configuration

The Kaggle dataset is organized into separate folders for each composer. Only the folders for Bach, Beethoven, Chopin, and Mozart will be used.

Standalone MIDI files located outside these four composer folders will not be included because their labels are not organized consistently with the selected composer folders.

```
In [ ]: # Define the path to the extracted Kaggle dataset.

DATASET_DIR = (
    Path.home()
    / "Desktop"
    / "archive"
    / "midiclassics"
)

# Define the folder where the processed data will be saved.
OUTPUT_DIR = (
    Path.home()
```

```

    / "Desktop"
    / "composer_classification_processed"
)

# Create the output directory when it does not already exist.
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

# Define the four composers required by the project instructions.
SELECTED_COMPOSERS = [
    "Bach",
    "Beethoven",
    "Chopin",
    "Mozart"
]

# Define supported MIDI file extensions.
SUPPORTED_EXTENSIONS = {
    ".mid",
    ".midi"
}

print("Dataset directory:")
print(DATASET_DIR)

print("\nOutput directory:")
print(OUTPUT_DIR)

print("\nSelected composers:")
print(SELECTED_COMPOSERS)

```

Dataset directory:
/Users/zinaothman/Desktop/archive/midiclassics

Output directory:
/Users/zinaothman/Desktop/composer_classification_processed

Selected composers:
['Bach', 'Beethoven', 'Chopin', 'Mozart']

2.1 Verify the Dataset Structure

The dataset path and the four required composer folders are checked before data collection begins.

```

In [ ]: # Confirm that the main dataset directory exists.

if not DATASET_DIR.exists():
    raise FileNotFoundError(
        f"The dataset directory was not found:\n{DATASET_DIR}\n\n"
        "Update DATASET_DIR so that it points to the midiclassics folder."
    )

# Check whether every required composer folder exists.
folder_status = {}

for composer in SELECTED_COMPOSERS:
    composer_directory = DATASET_DIR / composer
    folder_status[composer] = composer_directory.exists()

folder_status_df = pd.DataFrame({
    "composer": list(folder_status.keys()),
    "folder_found": list(folder_status.values())
})

display(folder_status_df)

if not all(folder_status.values()):
    missing_folders = [
        composer
        for composer, exists in folder_status.items()

```

```

        if not exists
    ]

    raise FileNotFoundError(
        f"The following composer folders were not found: {missing_folders}"
    )

print("All required composer folders were found.")

```

	composer	folder_found
0	Bach	True
1	Beethoven	True
2	Chopin	True
3	Mozart	True

All required composer folders were found.

3. Data Collection and Dataset Exploration

The project dataset is already provided as MIDI files. Data collection therefore consists of selecting all supported MIDI files located inside the four required composer folders and assigning each file its composer label.

```

In [ ]: def collect_composer_files(dataset_directory, composers):
    """
    Collect supported MIDI files from the selected composer folders.

    Parameters
    -----
    dataset_directory : pathlib.Path
        Root directory containing composer folders.
    composers : list of str
        Composer folder names included in the project.

    Returns
    -----
    pandas.DataFrame
        Inventory of selected MIDI files.
    """
    records = []

    for composer in composers:
        composer_directory = dataset_directory / composer

        for file_path in sorted(composer_directory.rglob("*")):
            if (
                file_path.is_file()
                and file_path.suffix.lower() in SUPPORTED_EXTENSIONS
            ):
                records.append({
                    "composer": composer.lower(),
                    "file_name": file_path.name,
                    "file_size_kb": round(
                        file_path.stat().st_size / 1024,
                        2
                    ),
                    "file_path": str(file_path)
                })

    return pd.DataFrame(records)

# Create the inventory for the four selected composers.
inventory_df = collect_composer_files(
    dataset_directory=DATASET_DIR,

```

```

    composers=SELECTED_COMPOSERS
)

if inventory_df.empty:
    raise ValueError(
        "No MIDI files were found in the selected composer folders."
    )

print("Total selected MIDI files:", len(inventory_df))

display(inventory_df.head())

```

Total selected MIDI files: 1630

	composer	file_name	file_size_kb	file_path
0	bach	AveMaria.mid	6.40	/Users/zinaothman/Desktop/archive/midiclassics...
1	bach	01 Menuet.mid	3.71	/Users/zinaothman/Desktop/archive/midiclassics...
2	bach	02 Menuet.mid	3.00	/Users/zinaothman/Desktop/archive/midiclassics...
3	bach	03 Menuet.mid	3.06	/Users/zinaothman/Desktop/archive/midiclassics...
4	bach	04 Menuet.mid	4.25	/Users/zinaothman/Desktop/archive/midiclassics...

3.2 Examine the Composer Distribution

The number of MIDI compositions available for each composer is calculated. This confirms that the four required classes were collected successfully and shows whether the dataset is balanced.

```

In [ ]: # Count the number of MIDI files for each composer.

composer_counts = (
    inventory_df["composer"]
    .value_counts()
    .reindex(
        [composer.lower() for composer in SELECTED_COMPOSERS]
    )
    .fillna(0)
    .astype(int)
)

composer_counts_df = (
    composer_counts
    .rename_axis("composer")
    .reset_index(name="number_of_files")
)

display(composer_counts_df)

print(
    "Beethoven MIDI files found:",
    int(composer_counts.get("beethoven", 0))
)

```

	composer	number_of_files
0	bach	1024
1	beethoven	213
2	chopin	136
3	mozart	257

Beethoven MIDI files found: 213

```

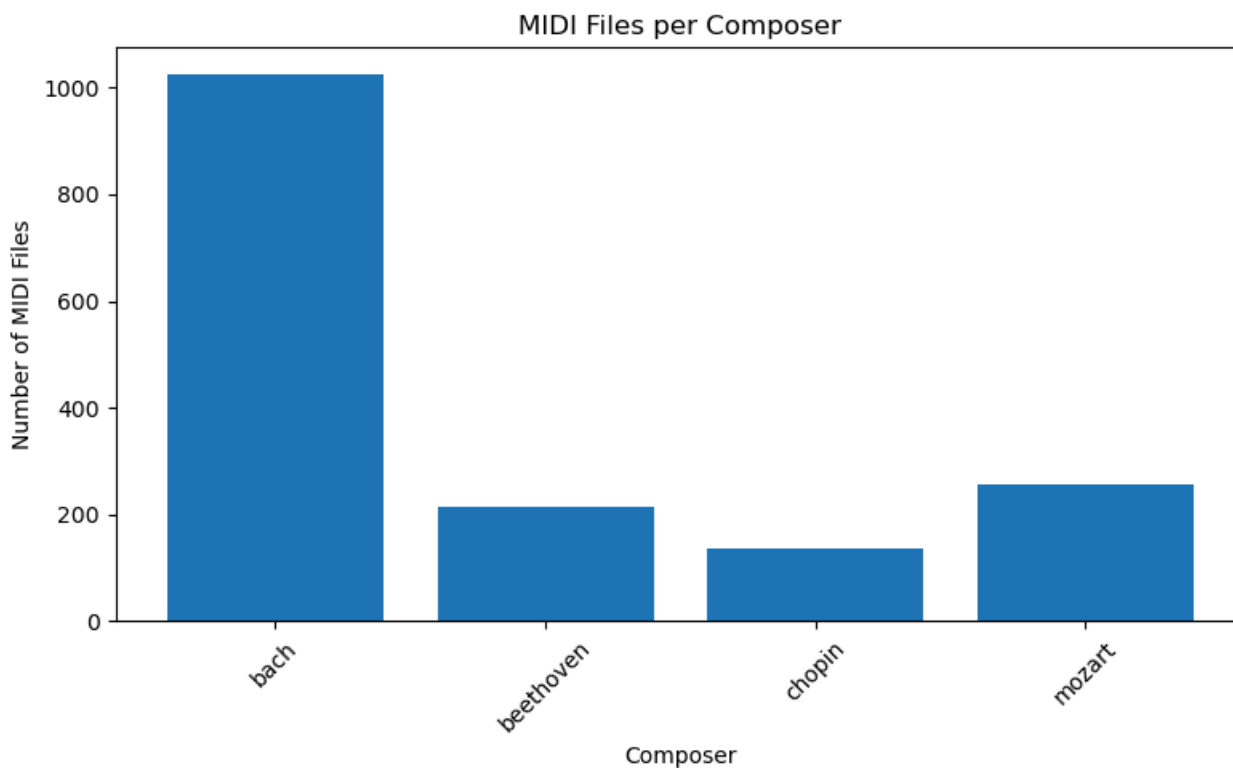
In [ ]: # Visualize the number of MIDI files for each composer.

```

```
plt.figure(figsize=(8, 5))

plt.bar(
    composer_counts.index,
    composer_counts.values
)

plt.title("MIDI Files per Composer")
plt.xlabel("Composer")
plt.ylabel("Number of MIDI Files")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



4. MIDI File Validation

Each selected MIDI file is checked before it is included in preprocessing. A usable MIDI file must:

- Open successfully with PrettyMIDI
- Contain at least one non-drum instrument
- Contain at least one pitched musical note

Drum tracks are excluded because they do not contain the melodic pitch information required for composer classification.

```
In [ ]: def validate_midi_file(file_path):
        """
        Check whether a MIDI file can be used for feature extraction.

        Parameters
        -----
        file_path : str
            Complete path to the MIDI file.

        Returns
        -----
        dict
            Validation status and basic MIDI statistics.
```

```

"""
try:
    midi_data = pretty_midi.PrettyMIDI(file_path)

    # Retain only non-drum instruments.
    non_drum_instruments = [
        instrument
        for instrument in midi_data.instruments
        if not instrument.is_drum
    ]

    # Count pitched notes across all non-drum instruments.
    number_of_notes = sum(
        len(instrument.notes)
        for instrument in non_drum_instruments
    )

    valid_midi = (
        len(non_drum_instruments) > 0
        and number_of_notes > 0
    )

    return {
        "valid_midi": valid_midi,
        "number_of_instruments": len(non_drum_instruments),
        "number_of_notes": number_of_notes,
        "duration_seconds": float(midi_data.get_end_time()),
        "error_message": None
    }

except Exception as error:
    return {
        "valid_midi": False,
        "number_of_instruments": 0,
        "number_of_notes": 0,
        "duration_seconds": 0.0,
        "error_message": str(error)
    }

```

In []: # Validate every MIDI file in the selected dataset.

```

validation_records = []

for index, row in inventory_df.iterrows():
    validation_result = validate_midi_file(
        row["file_path"]
    )

    validation_result["file_path"] = row["file_path"]
    validation_records.append(validation_result)

    if (index + 1) % 100 == 0:
        print(
            f"Validated {index + 1} of "
            f"{len(inventory_df)} files."
        )

validation_df = pd.DataFrame(validation_records)

# Combine validation results with the original inventory.
inventory_df = inventory_df.merge(
    validation_df,
    on="file_path",
    how="left"
)

print("MIDI validation completed.")

```

Validated 100 of 1630 files.
Validated 200 of 1630 files.
Validated 300 of 1630 files.
Validated 400 of 1630 files.
Validated 500 of 1630 files.
Validated 600 of 1630 files.
Validated 700 of 1630 files.
Validated 800 of 1630 files.
Validated 900 of 1630 files.
Validated 1000 of 1630 files.
Validated 1100 of 1630 files.
Validated 1200 of 1630 files.
Validated 1300 of 1630 files.
Validated 1400 of 1630 files.
Validated 1500 of 1630 files.
Validated 1600 of 1630 files.
MIDI validation completed.

4.1 Remove Unusable MIDI Files

Unreadable files and files without pitched notes are removed before the dataset is divided into training, validation, and testing sets.

```
In [ ]: # Separate usable and unusable MIDI files.

clean_inventory_df = inventory_df[
    inventory_df["valid_midi"]
].copy()

clean_inventory_df.reset_index(
    drop=True,
    inplace=True
)

problem_files_df = inventory_df[
    ~inventory_df["valid_midi"]
].copy()

print("Original MIDI files:", len(inventory_df))
print("Valid MIDI files:", len(clean_inventory_df))
print("Removed MIDI files:", len(problem_files_df))

if not problem_files_df.empty:
    display(
        problem_files_df[
            [
                "composer",
                "file_name",
                "error_message"
            ]
        ]
    )
```

Original MIDI files: 1630
Valid MIDI files: 1628
Removed MIDI files: 2

	composer	file_name	error_message
1028	beethoven	Anhang 14-3.mid	Could not decode key with 3 flats and mode 255
1494	mozart	K281 Piano Sonata n03 3mov.mid	Could not decode key with 2 flats and mode 2

5. Train, Validation, and Test Split

The valid compositions are divided into three subsets:

- 70% training
- 15% validation
- 15% testing

Stratified splitting is used so that each subset contains a proportional number of files from Bach, Beethoven, Chopin, and Mozart.

The original files are split before data augmentation. This prevents an original composition from appearing in one subset while an augmented version of the same composition appears in another subset.

```
In [ ]: # Create the 70% training set and a temporary 30% evaluation set.

train_inventory_df, temporary_inventory_df = train_test_split(
    clean_inventory_df,
    test_size=0.30,
    random_state=RANDOM_SEED,
    stratify=clean_inventory_df["composer"]
)

# Divide the temporary set equally into validation and testing sets.
dev_inventory_df, test_inventory_df = train_test_split(
    temporary_inventory_df,
    test_size=0.50,
    random_state=RANDOM_SEED,
    stratify=temporary_inventory_df["composer"]
)

# Create independent copies before adding split labels.
train_inventory_df = train_inventory_df.copy()
dev_inventory_df = dev_inventory_df.copy()
test_inventory_df = test_inventory_df.copy()

train_inventory_df["split"] = "train"
dev_inventory_df["split"] = "dev"
test_inventory_df["split"] = "test"

# Reset row indexes.
train_inventory_df.reset_index(drop=True, inplace=True)
dev_inventory_df.reset_index(drop=True, inplace=True)
test_inventory_df.reset_index(drop=True, inplace=True)

print("Training files:", len(train_inventory_df))
print("Validation files:", len(dev_inventory_df))
print("Testing files:", len(test_inventory_df))
```

Training files: 1139

Validation files: 244

Testing files: 245

5.1 Verify the Split Distribution

The distribution of composers is checked after splitting to confirm that every subset contains all four classes.

```
In [ ]: # Combine split records for distribution analysis.

split_inventory_df = pd.concat(
    [
        train_inventory_df,
        dev_inventory_df,
        test_inventory_df
    ],
    ignore_index=True
)

split_distribution = (
    split_inventory_df
    .groupby(["composer", "split"])
```

```

.size()
.unstack(fill_value=0)
.reindex(
    index=[
        "bach",
        "beethoven",
        "chopin",
        "mozart"
    ],
    columns=[
        "train",
        "dev",
        "test"
    ],
    fill_value=0
)

split_distribution["total"] = (
    split_distribution.sum(axis=1)
)

display(split_distribution)

```

	split	train	dev	test	total
composer					
bach		717	153	154	1024
beethoven		148	32	32	212
chopin		95	21	20	136
mozart		179	38	39	256

6. Data Preprocessing and Feature Extraction

Each MIDI composition is converted into an ordered sequence of musical note events.

The following features are extracted:

1. **Pitch:** MIDI pitch value of the note
2. **Duration:** Length of the note in seconds
3. **Velocity:** Strength or intensity of the note
4. **Time Shift:** Time between the current note onset and the previous note onset
5. **Tempo:** Tempo active when the note begins
6. **Chord Size:** Number of notes beginning at approximately the same time
7. **Chord Span:** Distance between the lowest and highest pitches in the chord

Chord size and chord span provide information about simultaneous notes and harmonic structure. A single note has a chord size of one and a chord span of zero.

In []: *# Define the order of features in each extracted event.*

```

FEATURE_NAMES = [
    "pitch",
    "duration",
    "velocity",
    "time_shift",
    "tempo",
    "chord_size",
    "chord_span"
]

```

```

NUMBER_OF_FEATURES = len(FEATURE_NAMES)

print("Extracted features:")
print(FEATURE_NAMES)

print("\nNumber of features per event:")
print(NUMBER_OF_FEATURES)

```

Extracted features:

```
['pitch', 'duration', 'velocity', 'time_shift', 'tempo', 'chord_size', 'chord_span']
```

Number of features per event:

7

```

In [ ]: def get_tempo_at_time(
        note_start,
        tempo_times,
        tempo_values
    ):
        """
        Return the tempo active at a specific note starting time.
        """
        if len(tempo_values) == 0:
            return 120.0

        tempo_index = np.searchsorted(
            tempo_times,
            note_start,
            side="right"
        ) - 1

        tempo_index = max(tempo_index, 0)

        return float(tempo_values[tempo_index])

```

```

In [ ]: def calculate_chord_information(notes, onset_precision=3):
        """
        Calculate chord size and pitch span for simultaneous notes.

        Notes with starting times that are equal after rounding are
        treated as belonging to the same chord.

        Parameters
        -----
        notes : list of dict
            Extracted note information.
        onset_precision : int
            Number of decimal places used to group note onsets.

        Returns
        -----
        dict
            Chord information indexed by rounded note onset.
        """
        onset_groups = defaultdict(list)

        for note in notes:
            rounded_onset = round(
                note["start"],
                onset_precision
            )

            onset_groups[rounded_onset].append(
                note["pitch"]
            )

        chord_information = {}

        for onset, pitches in onset_groups.items():
            chord_size = len(pitches)

```

```

    chord_span = (
        max(pitches) - min(pitches)
        if chord_size > 1
        else 0
    )

    chord_information[onset] = {
        "chord_size": chord_size,
        "chord_span": chord_span
    }

    return chord_information

```

```

In [ ]: def extract_midi_sequence(
    file_path,
    pitch_shift=0
):
    """
    Convert one MIDI composition into a numerical event sequence.

    Parameters
    -----
    file_path : str
        Complete path to the MIDI file.
    pitch_shift : int, default=0
        Number of semitones used for pitch-transposition
        augmentation.

    Returns
    -----
    numpy.ndarray
        Musical event sequence with seven features.
    """
    midi_data = pretty_midi.PrettyMIDI(file_path)

    # Retrieve tempo changes from the MIDI file.
    tempo_times, tempo_values = (
        midi_data.get_tempo_changes()
    )

    notes = []

    # Collect notes from every non-drum instrument.
    for instrument in midi_data.instruments:
        if instrument.is_drum:
            continue

        for note in instrument.notes:
            shifted_pitch = note.pitch + pitch_shift

            # Retain only valid MIDI pitch values.
            if 0 <= shifted_pitch <= 127:
                notes.append({
                    "pitch": int(shifted_pitch),
                    "start": float(note.start),
                    "end": float(note.end),
                    "velocity": int(note.velocity)
                })

    # Sort notes by starting time and then by pitch.
    notes.sort(
        key=lambda item: (
            item["start"],
            item["pitch"]
        )
    )

    # Determine which notes begin together as chords.
    chord_information = calculate_chord_information(

```

```

    notes
)

sequence = []
previous_start = 0.0

for note in notes:
    duration = max(
        note["end"] - note["start"],
        0.0
    )

    time_shift = max(
        note["start"] - previous_start,
        0.0
    )

    tempo = get_tempo_at_time(
        note_start=note["start"],
        tempo_times=tempo_times,
        tempo_values=tempo_values
    )

    rounded_onset = round(
        note["start"],
        3
    )

    chord_size = chord_information[
        rounded_onset
    ]["chord_size"]

    chord_span = chord_information[
        rounded_onset
    ]["chord_span"]

    sequence.append([
        float(note["pitch"]),
        float(duration),
        float(note["velocity"]),
        float(time_shift),
        float(tempo),
        float(chord_size),
        float(chord_span)
    ])

    previous_start = note["start"]

return np.asarray(
    sequence,
    dtype=np.float32
)

```

6.1 Test Feature Extraction

One valid MIDI composition is processed before the complete dataset to confirm that all seven musical features are extracted correctly.

```

In [ ]: # Select one valid training file.

sample_file_path = train_inventory_df.iloc[0][
    "file_path"
]

sample_sequence = extract_midi_sequence(
    file_path=sample_file_path,
    pitch_shift=0
)

```

```
print("Sample file:")
print(sample_file_path)

print("\nSample sequence shape:")
print(sample_sequence.shape)

display(
    pd.DataFrame(
        sample_sequence[:10],
        columns=FEATURE_NAMES
    )
)
```

Sample file:
/Users/zinaothman/Desktop/archive/midiclassics/Bach/Bwv001- 400 Chorales/039000b_.mid

Sample sequence shape:
(430, 7)

	pitch	duration	velocity	time_shift	tempo	chord_size	chord_span
0	60.0	0.545454	96.0	1.090908	110.000107	4.0	12.0
1	64.0	0.545454	96.0	0.000000	110.000107	4.0	12.0
2	67.0	0.545454	96.0	0.000000	110.000107	4.0	12.0
3	72.0	0.545454	96.0	0.000000	110.000107	4.0	12.0
4	57.0	1.090908	96.0	0.545454	110.000107	4.0	15.0
5	64.0	0.545454	96.0	0.000000	110.000107	4.0	15.0
6	69.0	1.090908	96.0	0.000000	110.000107	4.0	15.0
7	72.0	1.090908	96.0	0.000000	110.000107	4.0	15.0
8	65.0	0.545454	96.0	0.545454	110.000107	1.0	0.0
9	52.0	0.545454	96.0	0.545454	110.000107	4.0	19.0

7. Data Augmentation

Pitch transposition is used to increase the diversity of the training set. This method moves the pitches of a composition upward or downward while preserving its rhythm, tempo, intervals, and overall musical structure.

Each original training composition is retained, and two additional versions are created:

- Two semitones lower
- Original pitch
- Two semitones higher

Augmentation is applied only to the training set. The validation and testing sets remain unchanged to provide an unbiased evaluation of the final models.

```
In [ ]: # Define the pitch-transposition settings.

TRAIN_PITCH_SHIFTS = [
    -2,
    0,
    2
]

# Validation and testing compositions remain unchanged.
EVALUATION_PITCH_SHIFTS = [
    0
]
```

```
print("Training pitch shifts:", TRAIN_PITCH_SHIFTS)
print("Evaluation pitch shifts:", EVALUATION_PITCH_SHIFTS)
```

Training pitch shifts: [-2, 0, 2]
 Evaluation pitch shifts: [0]

```
In [ ]: def process_dataset_split(
        split_inventory,
        split_name,
        pitch_shifts
    ):
        """
        Extract sequences and labels from one dataset split.

        Parameters
        -----
        split_inventory : pandas.DataFrame
            MIDI files belonging to one dataset split.
        split_name : str
            Name of the dataset split.
        pitch_shifts : list of int
            Transpositions applied to each composition.

        Returns
        -----
        tuple
            Sequences, text labels, metadata, and errors.
        """
        sequences = []
        text_labels = []
        metadata_records = []
        error_records = []

        total_operations = (
            len(split_inventory)
            * len(pitch_shifts)
        )

        completed_operations = 0

        for _, row in split_inventory.iterrows():
            for pitch_shift in pitch_shifts:
                try:
                    sequence = extract_midi_sequence(
                        file_path=row["file_path"],
                        pitch_shift=pitch_shift
                    )

                    if len(sequence) == 0:
                        raise ValueError(
                            "No musical events were extracted."
                        )

                    sequences.append(sequence)
                    text_labels.append(row["composer"])

                    metadata_records.append({
                        "split": split_name,
                        "composer": row["composer"],
                        "file_name": row["file_name"],
                        "file_path": row["file_path"],
                        "pitch_shift": pitch_shift,
                        "augmented": pitch_shift != 0,
                        "sequence_length": len(sequence)
                    })

                except Exception as error:
                    error_records.append({
                        "split": split_name,
                        "composer": row["composer"],
```

```

        "file_name": row["file_name"],
        "pitch_shift": pitch_shift,
        "error_message": str(error)
    })

    completed_operations += 1

    if completed_operations % 100 == 0:
        print(
            f"{split_name}: processed "
            f"{completed_operations} of "
            f"{total_operations} operations."
        )

    return (
        sequences,
        text_labels,
        metadata_records,
        error_records
    )

```

7.1 Process the Training Data

The training data is processed using the original files and the two pitch-transposed versions.

```

In [ ]: (
    train_sequences,
    train_text_labels,
    train_metadata,
    train_errors
) = process_dataset_split(
    split_inventory=train_inventory_df,
    split_name="train",
    pitch_shifts=TRAIN_PITCH_SHIFTS
)

print("Generated training sequences:", len(train_sequences))
print("Training processing errors:", len(train_errors))

```



```

train: processed 100 of 3417 operations.
train: processed 200 of 3417 operations.
train: processed 300 of 3417 operations.
train: processed 400 of 3417 operations.
train: processed 500 of 3417 operations.
train: processed 600 of 3417 operations.
train: processed 700 of 3417 operations.
train: processed 800 of 3417 operations.
train: processed 900 of 3417 operations.
train: processed 1000 of 3417 operations.
train: processed 1100 of 3417 operations.
train: processed 1200 of 3417 operations.
train: processed 1300 of 3417 operations.
train: processed 1400 of 3417 operations.
train: processed 1500 of 3417 operations.
train: processed 1600 of 3417 operations.
train: processed 1700 of 3417 operations.
train: processed 1800 of 3417 operations.
train: processed 1900 of 3417 operations.
train: processed 2000 of 3417 operations.
train: processed 2100 of 3417 operations.
train: processed 2200 of 3417 operations.
train: processed 2300 of 3417 operations.
train: processed 2400 of 3417 operations.
train: processed 2500 of 3417 operations.
train: processed 2600 of 3417 operations.
train: processed 2700 of 3417 operations.
train: processed 2800 of 3417 operations.
train: processed 2900 of 3417 operations.
train: processed 3000 of 3417 operations.
train: processed 3100 of 3417 operations.
train: processed 3200 of 3417 operations.
train: processed 3300 of 3417 operations.
train: processed 3400 of 3417 operations.
Generated training sequences: 3417
Training processing errors: 0

```

7.2 Process the Validation Data

The validation data is processed without augmentation.

```

In [ ]: (
    dev_sequences,
    dev_text_labels,
    dev_metadata,
    dev_errors
) = process_dataset_split(
    split_inventory=dev_inventory_df,
    split_name="dev",
    pitch_shifts=EVALUATION_PITCH_SHIFTS
)

print("Generated validation sequences:", len(dev_sequences))
print("Validation processing errors:", len(dev_errors))

```

```

dev: processed 100 of 244 operations.
dev: processed 200 of 244 operations.
Generated validation sequences: 244
Validation processing errors: 0

```

7.3 Process the Testing Data

The testing data is processed without augmentation.

```

In [ ]: (
    test_sequences,
    test_text_labels,

```

```

    test_metadata,
    test_errors
) = process_dataset_split(
    split_inventory=test_inventory_df,
    split_name="test",
    pitch_shifts=EVALUATION_PITCH_SHIFTS
)

print("Generated testing sequences:", len(test_sequences))
print("Testing processing errors:", len(test_errors))

```

test: processed 100 of 245 operations.
test: processed 200 of 245 operations.
Generated testing sequences: 245
Testing processing errors: 0

In []: *# Combine processing errors from all three subsets.*

```

all_processing_errors = (
    train_errors
    + dev_errors
    + test_errors
)

processing_errors_df = pd.DataFrame(
    all_processing_errors
)

print(
    "Total feature-extraction errors:",
    len(processing_errors_df)
)

if not processing_errors_df.empty:
    display(processing_errors_df)

```

Total feature-extraction errors: 0

8. Composer Label Encoding

TensorFlow classification models require numerical target labels. The composer names are converted into integers using the following mapping:

- Bach: 0
- Beethoven: 1
- Chopin: 2
- Mozart: 3

The same mapping is applied to the training, validation, and testing subsets.

In []: *# Define the required composer label mapping explicitly.*

```

composer_to_index = {
    "bach": 0,
    "beethoven": 1,
    "chopin": 2,
    "mozart": 3
}

index_to_composer = {
    index: composer
    for composer, index
    in composer_to_index.items()
}

print("Composer label mapping:")

```

```
for composer, index in composer_to_index.items():
    print(f"{composer}: {index}")
```

Composer label mapping:

bach: 0
beethoven: 1
chopin: 2
mozart: 3

```
In [ ]: def encode_composer_labels(
        text_labels,
        label_mapping
    ):
        """
        Convert composer names into integer labels.
        """
        unknown_labels = sorted(
            set(text_labels)
            - set(label_mapping.keys())
        )

        if unknown_labels:
            raise ValueError(
                f"Unknown composer labels found: {unknown_labels}"
            )

        return np.asarray(
            [
                label_mapping[label]
                for label in text_labels
            ],
            dtype=np.int32
        )

# Encode composer labels for all dataset subsets.
y_train = encode_composer_labels(
    train_text_labels,
    composer_to_index
)

y_dev = encode_composer_labels(
    dev_text_labels,
    composer_to_index
)

y_test = encode_composer_labels(
    test_text_labels,
    composer_to_index
)

print("Training labels shape:", y_train.shape)
print("Validation labels shape:", y_dev.shape)
print("Testing labels shape:", y_test.shape)
```

Training labels shape: (3417,)
Validation labels shape: (244,)
Testing labels shape: (245,)

9. Sequence Normalization and Padding

The MIDI compositions contain different numbers of musical events. TensorFlow models require input samples with consistent dimensions.

The extracted numerical features are first normalized. The sequences are then padded or truncated to a fixed length using the TensorFlow/Keras `pad_sequences` function.

Post-padding adds zeros after the final musical event. Post-truncation removes events from the end of compositions that exceed the selected maximum sequence length.

9.1 Examine Training Sequence Lengths

The maximum sequence length is determined only from the training data. This prevents information from the validation or testing data from influencing preprocessing decisions.

In []: *# Calculate the length of every processed training sequence.*

```
train_sequence_lengths = np.asarray(
    [
        len(sequence)
        for sequence in train_sequences
    ]
)

sequence_length_summary = (
    pd.Series(train_sequence_lengths)
    .describe(
        percentiles=[
            0.50,
            0.75,
            0.90,
            0.95,
            0.99
        ]
    )
    .to_frame(
        name="training_sequence_length"
    )
)

display(sequence_length_summary)
```

training_sequence_length	
count	3417.000000
mean	3209.338016
std	4951.071924
min	83.000000
50%	1145.000000
75%	3967.000000
90%	8871.400000
95%	13063.400000
99%	21885.000000
max	46897.000000

The training dataset was analyzed to determine an appropriate maximum sequence length for padding. The initial estimate was based on the 95th percentile of the training sequence lengths. To keep memory usage and computational cost manageable during model training, the sequence length was capped at 2,000 musical events. Sequences shorter than 2,000 events are padded with zeros, while longer sequences are truncated after the first 2,000 events. This approach provides consistent input dimensions while maintaining efficient preprocessing and training.

In []: *# Select a practical sequence length based on the training data and an upper length cap.*

```
SEQUENCE_PERCENTILE = 95
SEQUENCE_LENGTH_CAP = 2000
```

```

percentile_sequence_length = int(
    np.percentile(
        train_sequence_lengths,
        SEQUENCE_PERCENTILE
    )
)

MAX_SEQUENCE_LENGTH = min(
    percentile_sequence_length,
    SEQUENCE_LENGTH_CAP
)

MAX_SEQUENCE_LENGTH = max(
    MAX_SEQUENCE_LENGTH,
    1
)

print(
    "95th percentile sequence length:",
    percentile_sequence_length
)

print(
    "Maximum sequence length used:",
    MAX_SEQUENCE_LENGTH
)

```

95th percentile sequence length: 13063

Maximum sequence length used: 2000

Although the 95th percentile of the training sequences was 13,063 events, an upper limit of 2,000 events was intentionally applied to reduce computational requirements and produce fixed-length inputs for the deep learning models.

9.2 Normalize Musical Features

The extracted features use different numerical scales. The following transformations place them in approximately the same zero-to-one range:

- Pitch divided by 127
- Duration clipped at 10 seconds and divided by 10
- Velocity divided by 127
- Time shift clipped at 10 seconds and divided by 10
- Tempo clipped at 300 BPM and divided by 300
- Chord size clipped at 20 notes and divided by 20
- Chord span divided by 127

```

In [ ]: def normalize_sequence(sequence):
        """
        Normalize the seven musical features in one sequence.

        Feature order:
        pitch, duration, velocity, time shift, tempo,
        chord size, chord span
        """
        normalized_sequence = sequence.astype(
            np.float32
        ).copy()

        # Normalize pitch.
        normalized_sequence[:, 0] = (
            np.clip(
                normalized_sequence[:, 0],
                0,
                127
            )
        )

```

```

        / 127.0
    )

    # Normalize note duration.
    normalized_sequence[:, 1] = (
        np.clip(
            normalized_sequence[:, 1],
            0,
            10
        )
        / 10.0
    )

    # Normalize note velocity.
    normalized_sequence[:, 2] = (
        np.clip(
            normalized_sequence[:, 2],
            0,
            127
        )
        / 127.0
    )

    # Normalize time shift.
    normalized_sequence[:, 3] = (
        np.clip(
            normalized_sequence[:, 3],
            0,
            10
        )
        / 10.0
    )

    # Normalize tempo.
    normalized_sequence[:, 4] = (
        np.clip(
            normalized_sequence[:, 4],
            0,
            300
        )
        / 300.0
    )

    # Normalize chord size.
    normalized_sequence[:, 5] = (
        np.clip(
            normalized_sequence[:, 5],
            0,
            20
        )
        / 20.0
    )

    # Normalize chord span.
    normalized_sequence[:, 6] = (
        np.clip(
            normalized_sequence[:, 6],
            0,
            127
        )
        / 127.0
    )

    return normalized_sequence

```

```

In [ ]: # Normalize all extracted musical sequences.

normalized_train_sequences = [
    normalize_sequence(sequence)
    for sequence in train_sequences

```

```

]

normalized_dev_sequences = [
    normalize_sequence(sequence)
    for sequence in dev_sequences
]

normalized_test_sequences = [
    normalize_sequence(sequence)
    for sequence in test_sequences
]

print("All sequences were normalized successfully.")

```

All sequences were normalized successfully.

9.3 Apply TensorFlow/Keras Padding

The normalized sequences are converted into fixed-length, three-dimensional arrays:

number of compositions × maximum sequence length × number of features

In []: *# Pad and truncate the sequences using TensorFlow/Keras.*

```

X_train = pad_sequences(
    normalized_train_sequences,
    maxlen=MAX_SEQUENCE_LENGTH,
    dtype="float32",
    padding="post",
    truncating="post",
    value=0.0
)

X_dev = pad_sequences(
    normalized_dev_sequences,
    maxlen=MAX_SEQUENCE_LENGTH,
    dtype="float32",
    padding="post",
    truncating="post",
    value=0.0
)

X_test = pad_sequences(
    normalized_test_sequences,
    maxlen=MAX_SEQUENCE_LENGTH,
    dtype="float32",
    padding="post",
    truncating="post",
    value=0.0
)

print("Training sequence shape:", X_train.shape)
print("Validation sequence shape:", X_dev.shape)
print("Testing sequence shape:", X_test.shape)

```

Training sequence shape: (3417, 2000, 7)
 Validation sequence shape: (244, 2000, 7)
 Testing sequence shape: (245, 2000, 7)

In []: *# Confirm that each sequence has one matching composer Label.*

```

assert X_train.shape[0] == y_train.shape[0]
assert X_dev.shape[0] == y_dev.shape[0]
assert X_test.shape[0] == y_test.shape[0]

# Confirm that all sequences contain seven features.
assert X_train.shape[2] == NUMBER_OF_FEATURES
assert X_dev.shape[2] == NUMBER_OF_FEATURES
assert X_test.shape[2] == NUMBER_OF_FEATURES

```

```
print("All sequences and labels are correctly aligned.")
```

All sequences and labels are correctly aligned.

10. TensorFlow Dataset Preparation

TensorFlow datasets are created from the processed arrays to confirm that the data can be passed directly into the LSTM and CNN models.

The training dataset is shuffled. The validation and testing datasets remain in their existing order.

In []: *# Define the batch size used to test TensorFlow compatibility.*

```
BATCH_SIZE = 32

train_dataset = (
    tf.data.Dataset
        .from_tensor_slices(
            (X_train, y_train)
        )
        .shuffle(
            buffer_size=len(X_train),
            seed=RANDOM_SEED,
            reshuffle_each_iteration=True
        )
        .batch(BATCH_SIZE)
        .prefetch(tf.data.AUTOTUNE)
)

dev_dataset = (
    tf.data.Dataset
        .from_tensor_slices(
            (X_dev, y_dev)
        )
        .batch(BATCH_SIZE)
        .prefetch(tf.data.AUTOTUNE)
)

test_dataset = (
    tf.data.Dataset
        .from_tensor_slices(
            (X_test, y_test)
        )
        .batch(BATCH_SIZE)
        .prefetch(tf.data.AUTOTUNE)
)

print("TensorFlow datasets created successfully.")
```

TensorFlow datasets created successfully.

In []: *# Inspect one batch from the TensorFlow training dataset.*

```
for sequence_batch, label_batch in train_dataset.take(1):
    print(
        "Sequence batch shape:",
        sequence_batch.shape
    )

    print(
        "Label batch shape:",
        label_batch.shape
    )

    print(
        "Sequence data type:",
        sequence_batch.dtype
```



```
)

print(
    "Label data type:",
    label_batch.dtype
)

```

Sequence batch shape: (32, 2000, 7)
 Label batch shape: (32,)
 Sequence data type: <dtype: 'float32'>
 Label data type: <dtype: 'int32'>

2026-07-21 21:49:51.190463: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence

11. Save the Processed Data

The TensorFlow-ready sequences, numerical composer labels, metadata, label mapping, and preprocessing configuration are saved for the LSTM and CNN model-development stages.

```
In [ ]: # Save the processed sequence and label arrays.

np.save(
    OUTPUT_DIR / "X_train.npy",
    X_train
)

np.save(
    OUTPUT_DIR / "y_train.npy",
    y_train
)

np.save(
    OUTPUT_DIR / "X_dev.npy",
    X_dev
)

np.save(
    OUTPUT_DIR / "y_dev.npy",
    y_dev
)

np.save(
    OUTPUT_DIR / "X_test.npy",
    X_test
)

np.save(
    OUTPUT_DIR / "y_test.npy",
    y_test
)

print("Processed sequence and label arrays were saved.")

```

Processed sequence and label arrays were saved.

```
In [ ]: # Save the composer-to-integer label mapping.

with open(
    OUTPUT_DIR / "composer_labels.json",
    "w",
    encoding="utf-8"
) as label_file:
    json.dump(
        composer_to_index,
        label_file,
        indent=4
    )

```

```
print("Composer label mapping was saved.")
```

Composer label mapping was saved.

```
In [ ]: # Save the preprocessing configuration.

processing_configuration = {
    "framework": "TensorFlow",
    "selected_composers": [
        "bach",
        "beethoven",
        "chopin",
        "mozart"
    ],
    "data_split": {
        "training": 0.70,
        "validation": 0.15,
        "testing": 0.15
    },
    "feature_names": FEATURE_NAMES,
    "number_of_features": NUMBER_OF_FEATURES,
    "maximum_sequence_length": MAX_SEQUENCE_LENGTH,
    "sequence_percentile": SEQUENCE_PERCENTILE,
    "sequence_length_cap": SEQUENCE_LENGTH_CAP,
    "training_pitch_shifts": TRAIN_PITCH_SHIFTS,
    "evaluation_pitch_shifts": EVALUATION_PITCH_SHIFTS,
    "padding": "post",
    "truncating": "post",
    "padding_value": 0.0,
    "batch_size": BATCH_SIZE
}

with open(
    OUTPUT_DIR / "processing_configuration.json",
    "w",
    encoding="utf-8"
) as configuration_file:
    json.dump(
        processing_configuration,
        configuration_file,
        indent=4
    )

print("Preprocessing configuration was saved.")
```

Preprocessing configuration was saved.

```
In [ ]: # Combine metadata from all processed sequences.

all_sequence_metadata = (
    train_metadata
    + dev_metadata
    + test_metadata
)

sequence_metadata_df = pd.DataFrame(
    all_sequence_metadata
)

# Add the numerical composer labels.
sequence_metadata_df["composer_index"] = (
    sequence_metadata_df["composer"]
    .map(composer_to_index)
)

# Save the sequence metadata.
sequence_metadata_df.to_csv(
    OUTPUT_DIR / "sequence_metadata.csv",
    index=False
)
```

```
# Save the original split inventory.
split_inventory_df.to_csv(
    OUTPUT_DIR / "dataset_split_inventory.csv",
    index=False
)

# Save feature-extraction errors when they exist.
if not processing_errors_df.empty:
    processing_errors_df.to_csv(
        OUTPUT_DIR / "processing_errors.csv",
        index=False
    )

print("Sequence metadata and split inventory were saved.")

display(sequence_metadata_df.head())
```

Sequence metadata and split inventory were saved.

	split	composer	file_name	file_path	pitch_shift	augmented	sequence_length
0	train	bach	039000b_mid	/Users/zinaothman/Desktop/archive/midiclassics...	-2	True	431
1	train	bach	039000b_mid	/Users/zinaothman/Desktop/archive/midiclassics...	0	False	431
2	train	bach	039000b_mid	/Users/zinaothman/Desktop/archive/midiclassics...	2	True	431
3	train	bach	024864b6.mid	/Users/zinaothman/Desktop/archive/midiclassics...	-2	True	358
4	train	bach	024864b6.mid	/Users/zinaothman/Desktop/archive/midiclassics...	0	False	358

12. Final Validation

The saved arrays are loaded again to confirm that the processed files are complete and ready for use by the TensorFlow LSTM and CNN models.

In []: *# Reload all saved sequence and label arrays.*

```
saved_X_train = np.load(
    OUTPUT_DIR / "X_train.npy"
)

saved_y_train = np.load(
    OUTPUT_DIR / "y_train.npy"
)

saved_X_dev = np.load(
    OUTPUT_DIR / "X_dev.npy"
)

saved_y_dev = np.load(
    OUTPUT_DIR / "y_dev.npy"
)

saved_X_test = np.load(
    OUTPUT_DIR / "X_test.npy"
)

saved_y_test = np.load(
    OUTPUT_DIR / "y_test.npy"
)

print("Saved arrays loaded successfully.")
```

Saved arrays loaded successfully.

```
In [ ]: # Verify sequence and label alignment.

assert saved_X_train.shape[0] == saved_y_train.shape[0]
assert saved_X_dev.shape[0] == saved_y_dev.shape[0]
assert saved_X_test.shape[0] == saved_y_test.shape[0]

# Verify sequence dimensions.
assert saved_X_train.shape[1] == MAX_SEQUENCE_LENGTH
assert saved_X_dev.shape[1] == MAX_SEQUENCE_LENGTH
assert saved_X_test.shape[1] == MAX_SEQUENCE_LENGTH

# Verify feature dimensions.
assert saved_X_train.shape[2] == NUMBER_OF_FEATURES
assert saved_X_dev.shape[2] == NUMBER_OF_FEATURES
assert saved_X_test.shape[2] == NUMBER_OF_FEATURES

# Verify that all feature values are finite.
assert np.isfinite(saved_X_train).all()
assert np.isfinite(saved_X_dev).all()
assert np.isfinite(saved_X_test).all()

# Verify that normalized feature values remain in the expected range.
assert saved_X_train.min() >= 0.0
assert saved_X_train.max() <= 1.0

assert saved_X_dev.min() >= 0.0
assert saved_X_dev.max() <= 1.0

assert saved_X_test.min() >= 0.0
assert saved_X_test.max() <= 1.0

# Verify composer label values.
assert saved_y_train.min() >= 0
assert saved_y_train.max() <= 3

assert saved_y_dev.min() >= 0
assert saved_y_dev.max() <= 3

assert saved_y_test.min() >= 0
assert saved_y_test.max() <= 3

print("All final validation checks passed.")
```

All final validation checks passed.

```
In [ ]: # Display the final processed dataset summary.

final_dataset_summary = pd.DataFrame({
    "dataset_split": [
        "Training",
        "Validation",
        "Testing"
    ],
    "number_of_sequences": [
        len(saved_X_train),
        len(saved_X_dev),
        len(saved_X_test)
    ],
    "sequence_length": [
        saved_X_train.shape[1],
        saved_X_dev.shape[1],
        saved_X_test.shape[1]
    ],
    "features_per_event": [
        saved_X_train.shape[2],
        saved_X_dev.shape[2],
        saved_X_test.shape[2]
    ],
    "number_of_classes": [
        len(np.unique(saved_y_train)),
```

```
        len(np.unique(saved_y_dev)),
        len(np.unique(saved_y_test))
    ]
})

display(final_dataset_summary)

print("Feature order:")
print(FEATURE_NAMES)

print("\nComposer label mapping:")
print(composer_to_index)

print("\nProcessed files saved in:")
print(OUTPUT_DIR)
```

	dataset_split	number_of_sequences	sequence_length	features_per_event	number_of_classes
0	Training	3417	2000	7	4
1	Validation	244	2000	7	4
2	Testing	245	2000	7	4

Feature order:
['pitch', 'duration', 'velocity', 'time_shift', 'tempo', 'chord_size', 'chord_span']

Composer label mapping:
{'bach': 0, 'beethoven': 1, 'chopin': 2, 'mozart': 3}

Processed files saved in:
/Users/zinaothman/Desktop/composer_classification_processed

13. Data Preparation Summary

The preprocessing pipeline successfully prepared the MIDI dataset for deep learning by validating the input files, applying data augmentation to the training set, extracting musical features, encoding composer labels, and normalizing the feature values. TensorFlow/Keras padding and truncation were applied to create fixed-length input sequences of 2,000 musical events. Sequences shorter than 2,000 events were padded with zeros, while longer sequences were truncated after the first 2,000 events. This ensured consistent input dimensions for the LSTM and CNN models while keeping memory usage and computational requirements manageable. The processed datasets, metadata, and configuration files were saved and validated, providing a reproducible, model-ready dataset for the subsequent training and evaluation stages of the project.

Dataset Limitation The dataset is imbalanced, with Bach containing substantially more compositions than Beethoven, Chopin, and Mozart. Stratified train, validation, and test splits were used to preserve the original class distribution across each dataset. While no balancing techniques were applied during preprocessing, the LSTM and CNN models may benefit from strategies such as class weighting or other imbalance-aware methods during training to reduce potential bias toward the majority class.

Part II — LSTM Modeling, Training, and Evaluation

14. LSTM Model Development

Using the preprocessed sequence data prepared earlier in the project section, we created an LSTM model for composer classification.

The goal of this LSTM model development is to load the processed sequence arrays, examine the input dimensions, address class imbalance, then build a baseline LSTM model, train and tune the model, and evaluate performance using

accuracy, precision, F1-score, and recall.

14.1 Prepare Processed Data (Output From Part I) for LSTM Modeling

```
In [ ]: !unzip -o /content/composer_classification_processed.zip -d /content/
```

```
Archive: /content/composer_classification_processed.zip
  inflating: /content/composer_classification_processed/X_dev.npy
  inflating: /content/composer_classification_processed/processing_configuration.json
  inflating: /content/composer_classification_processed/y_dev.npy
  inflating: /content/composer_classification_processed/sequence_metadata.csv
  inflating: /content/composer_classification_processed/composer_labels.json
  inflating: /content/composer_classification_processed/dataset_split_inventory.csv
  inflating: /content/composer_classification_processed/y_train.npy
  inflating: /content/composer_classification_processed/y_test.npy
  inflating: /content/composer_classification_processed/X_test.npy
  inflating: /content/composer_classification_processed/X_train.npy
```

```
In [ ]: from pathlib import Path
import os

PROCESSED_DIR = Path("/content/composer_classification_processed")

required_files = [
    "X_train.npy",
    "y_train.npy",
    "X_dev.npy",
    "y_dev.npy",
    "X_test.npy",
    "y_test.npy",
    "composer_labels.json",
    "processing_configuration.json"
]

print("Processed directory exists:", PROCESSED_DIR.exists())
print()

missing_files = []
for file_name in required_files:
    file_path = PROCESSED_DIR / file_name
    exists = file_path.exists()
    print(f"{file_name}: {'FOUND' if exists else 'MISSING'}")
    if not exists:
        missing_files.append(file_name)

if missing_files:
    print("\nMissing files:", missing_files)
else:
    print("\nAll required processed files are present.")
```

Processed directory exists: True

```
X_train.npy: FOUND
y_train.npy: FOUND
X_dev.npy: FOUND
y_dev.npy: FOUND
X_test.npy: FOUND
y_test.npy: FOUND
composer_labels.json: FOUND
processing_configuration.json: FOUND
```

All required processed files are present.

```
In [ ]: import numpy as np
import json

X_train = np.load(PROCESSED_DIR / "X_train.npy").astype(np.float32)
y_train = np.load(PROCESSED_DIR / "y_train.npy").reshape(-1).astype(np.int32)
```

```

X_dev = np.load(PROCESSED_DIR / "X_dev.npy").astype(np.float32)
y_dev = np.load(PROCESSED_DIR / "y_dev.npy").reshape(-1).astype(np.int32)

X_test = np.load(PROCESSED_DIR / "X_test.npy").astype(np.float32)
y_test = np.load(PROCESSED_DIR / "y_test.npy").reshape(-1).astype(np.int32)

with open(PROCESSED_DIR / "composer_labels.json", "r") as f:
    composer_labels = json.load(f)

with open(PROCESSED_DIR / "processing_configuration.json", "r") as f:
    processing_config = json.load(f)

print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_dev shape:", X_dev.shape)
print("y_dev shape:", y_dev.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
print("Composer labels:", composer_labels)

```

```

X_train shape: (3417, 2000, 7)
y_train shape: (3417,)
X_dev shape: (244, 2000, 7)
y_dev shape: (244,)
X_test shape: (245, 2000, 7)
y_test shape: (245,)
Composer labels: {'bach': 0, 'beethoven': 1, 'chopin': 2, 'mozart': 3}

```

14.2 Import Model Training Libraries

```

In [ ]: import random
import sys

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
import sklearn

from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Masking, LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau

```

14.3 Define the Processed Data Folder

```

In [ ]: PROCESSED_DIR = Path("/content/composer_classification_processed")
print("Processed data folder:", PROCESSED_DIR)

```

Processed data folder: /content/composer_classification_processed

14.4 Check for Required Processed Files

```

In [ ]: required_files = [
    "X_train.npy",
    "y_train.npy",
    "X_dev.npy",

```

```

    "y_dev.npy",
    "X_test.npy",
    "y_test.npy",
    "composer_labels.json",
    "processing_configuration.json"
]

print("Processed directory exists:", PROCESSED_DIR.exists())
print()

missing_files = []
for file_name in required_files:
    file_path = PROCESSED_DIR / file_name
    exists = file_path.exists()
    print(f"{file_name}: {'FOUND' if exists else 'MISSING'}")
    if not exists:
        missing_files.append(file_name)

if missing_files:
    print("\nMissing files:", missing_files)
else:
    print("\nAll required processed files are present.")

```

Processed directory exists: True

X_train.npy: FOUND
 y_train.npy: FOUND
 X_dev.npy: FOUND
 y_dev.npy: FOUND
 X_test.npy: FOUND
 y_test.npy: FOUND
 composer_labels.json: FOUND
 processing_configuration.json: FOUND

All required processed files are present.

14.5 Load the Processed Sequence Arrays

```

In [ ]: X_train = np.load(PROCESSED_DIR / "X_train.npy").astype(np.float32)
        y_train = np.load(PROCESSED_DIR / "y_train.npy").reshape(-1).astype(np.int32)

        X_dev = np.load(PROCESSED_DIR / "X_dev.npy").astype(np.float32)
        y_dev = np.load(PROCESSED_DIR / "y_dev.npy").reshape(-1).astype(np.int32)

        X_test = np.load(PROCESSED_DIR / "X_test.npy").astype(np.float32)
        y_test = np.load(PROCESSED_DIR / "y_test.npy").reshape(-1).astype(np.int32)

        with open(PROCESSED_DIR / "composer_labels.json", "r") as f:
            composer_labels = json.load(f)

        with open(PROCESSED_DIR / "processing_configuration.json", "r") as f:
            processing_config = json.load(f)

        print("X_train shape:", X_train.shape)
        print("y_train shape:", y_train.shape)
        print("X_dev shape:", X_dev.shape)
        print("y_dev shape:", y_dev.shape)
        print("X_test shape:", X_test.shape)
        print("y_test shape:", y_test.shape)
        print("Composer labels:", composer_labels)

```

X_train shape: (3417, 2000, 7)
 y_train shape: (3417,)
 X_dev shape: (244, 2000, 7)
 y_dev shape: (244,)
 X_test shape: (245, 2000, 7)
 y_test shape: (245,)
 Composer labels: {'bach': 0, 'beethoven': 1, 'chopin': 2, 'mozart': 3}

14.6 Perform Data Sanity Checks

```
In [ ]: print("Training dtype:", X_train.dtype, y_train.dtype)
        print("Validation dtype:", X_dev.dtype, y_dev.dtype)
        print("Test dtype:", X_test.dtype, y_test.dtype)

        print("Any NaNs in X_train:", np.isnan(X_train).any())
        print("Any NaNs in X_dev:", np.isnan(X_dev).any())
        print("Any NaNs in X_test:", np.isnan(X_test).any())

        print("Unique training labels:", np.unique(y_train))
        print("Unique validation labels:", np.unique(y_dev))
        print("Unique test labels:", np.unique(y_test))
```

```
Training dtype: float32 int32
Validation dtype: float32 int32
Test dtype: float32 int32
Any NaNs in X_train: False
Any NaNs in X_dev: False
Any NaNs in X_test: False
Unique training labels: [0 1 2 3]
Unique validation labels: [0 1 2 3]
Unique test labels: [0 1 2 3]
```

14.7 Inspect Class Distribution and Compute Class Weights

```
In [ ]: class_counts = pd.Series(y_train).value_counts().sort_index()
        print("Training class counts:")
        print(class_counts)

        class_names = [k for k, v in sorted(composer_labels.items(), key=lambda x: x[1])]
        print("\nClass index to composer mapping:")
        for idx, name in enumerate(class_names):
            print(f"{idx}: {name}")

        classes = np.unique(y_train)
        weights = compute_class_weight(
            class_weight="balanced",
            classes=classes,
            y=y_train
        )

        class_weight_dict = {int(cls): float(w) for cls, w in zip(classes, weights)}

        print("\nComputed class weights:")
        for cls, weight in class_weight_dict.items():
            print(f"Class {cls} ({class_names[cls]}): {weight:.4f}")
```

```
Training class counts:
0    2151
1     444
2     285
3     537
Name: count, dtype: int64
```

```
Class index to composer mapping:
0: bach
1: beethoven
2: chopin
3: mozart
```

```
Computed class weights:
Class 0 (bach): 0.3971
Class 1 (beethoven): 1.9240
Class 2 (chopin): 2.9974
Class 3 (mozart): 1.5908
```

14.8 Set Random Seeds for Reproducibility

```
In [ ]: SEED = 42

random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("Random seeds set to:", SEED)
```

Random seeds set to: 42

14.9 Build the Baseline LSTM Model

```
In [ ]: num_timesteps = X_train.shape[1]
num_features = X_train.shape[2]
num_classes = len(np.unique(y_train))

print("Timesteps:", num_timesteps)
print("Features:", num_features)
print("Classes:", num_classes)

baseline_lstm = Sequential([
    Input(shape=(num_timesteps, num_features)),
    Masking(mask_value=0.0),
    LSTM(64),
    Dropout(0.3),
    Dense(64, activation="relu"),
    Dropout(0.3),
    Dense(num_classes, activation="softmax")
])

baseline_lstm.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

baseline_lstm.summary()
```

Timesteps: 2000

Features: 7

Classes: 4

Model: "sequential"

Layer (type)	Output Shape	Param #
masking (Masking)	(None, 2000, 7)	0
lstm (LSTM)	(None, 64)	18,432
dropout (Dropout)	(None, 64)	0
dense (Dense)	(None, 64)	4,160
dropout_1 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 4)	260

Total params: 22,852 (89.27 KB)

Trainable params: 22,852 (89.27 KB)

Non-trainable params: 0 (0.00 B)

15. LSTM Training and Hyperparameter Tuning

In this section, we train the baseline LSTM model. We then test a tuned LSTM configuration to improve performance.

15.1 Define Baseline Training Callbacks

```
In [ ]: baseline_checkpoint_path = PROCESSED_DIR / "baseline_lstm_best.keras"

baseline_callbacks = [
    EarlyStopping(
        monitor="val_loss",
        patience=5,
        restore_best_weights=True
    ),
    ModelCheckpoint(
        filepath=str(baseline_checkpoint_path),
        monitor="val_loss",
        save_best_only=True
    ),
    ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=3,
        min_lr=1e-6
    )
]

print("Baseline checkpoint path:", baseline_checkpoint_path)
```

Baseline checkpoint path: /content/composer_classification_processed/baseline_lstm_best.keras

15.2 Train the Baseline LSTM Model

```
In [ ]: baseline_history = baseline_lstm.fit(
    X_train,
    y_train,
    validation_data=(X_dev, y_dev),
    epochs=20,
    batch_size=32,
    class_weight=class_weight_dict,
    callbacks=baseline_callbacks,
    verbose=1
)
```

```

Epoch 1/20
107/107 ————— 228s 2s/step - accuracy: 0.5048 - loss: 1.3423 - val_accuracy: 0.5779 - val_loss: 1.2387 - learning_rate: 0.0010
Epoch 2/20
107/107 ————— 269s 3s/step - accuracy: 0.5830 - loss: 1.2595 - val_accuracy: 0.5574 - val_loss: 1.1761 - learning_rate: 0.0010
Epoch 3/20
107/107 ————— 320s 3s/step - accuracy: 0.5938 - loss: 1.2151 - val_accuracy: 0.5697 - val_loss: 1.1813 - learning_rate: 0.0010
Epoch 4/20
107/107 ————— 273s 3s/step - accuracy: 0.5982 - loss: 1.2038 - val_accuracy: 0.5656 - val_loss: 1.1727 - learning_rate: 0.0010
Epoch 5/20
107/107 ————— 261s 2s/step - accuracy: 0.6207 - loss: 1.1729 - val_accuracy: 0.5984 - val_loss: 1.1138 - learning_rate: 0.0010
Epoch 6/20
107/107 ————— 267s 2s/step - accuracy: 0.6172 - loss: 1.1435 - val_accuracy: 0.6230 - val_loss: 1.0328 - learning_rate: 0.0010
Epoch 7/20
107/107 ————— 324s 2s/step - accuracy: 0.6336 - loss: 1.1064 - val_accuracy: 0.5697 - val_loss: 1.0823 - learning_rate: 0.0010
Epoch 8/20
107/107 ————— 315s 2s/step - accuracy: 0.6236 - loss: 1.0844 - val_accuracy: 0.5943 - val_loss: 1.0106 - learning_rate: 0.0010
Epoch 9/20
107/107 ————— 266s 2s/step - accuracy: 0.6160 - loss: 1.0695 - val_accuracy: 0.5779 - val_loss: 1.0671 - learning_rate: 0.0010
Epoch 10/20
107/107 ————— 325s 2s/step - accuracy: 0.6310 - loss: 1.0433 - val_accuracy: 0.5738 - val_loss: 1.0895 - learning_rate: 0.0010
Epoch 11/20
107/107 ————— 310s 2s/step - accuracy: 0.5988 - loss: 1.0613 - val_accuracy: 0.3361 - val_loss: 1.2173 - learning_rate: 0.0010
Epoch 12/20
107/107 ————— 253s 2s/step - accuracy: 0.5771 - loss: 1.1356 - val_accuracy: 0.6066 - val_loss: 1.1026 - learning_rate: 5.0000e-04
Epoch 13/20
107/107 ————— 265s 2s/step - accuracy: 0.6409 - loss: 1.0624 - val_accuracy: 0.6025 - val_loss: 1.0736 - learning_rate: 5.0000e-04

```

15.3 Plot and Save Baseline Training History

```

In [ ]: baseline_history_df = pd.DataFrame(baseline_history.history)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(baseline_history_df["loss"], label="Train Loss")
plt.plot(baseline_history_df["val_loss"], label="Validation Loss")
plt.title("Baseline LSTM Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

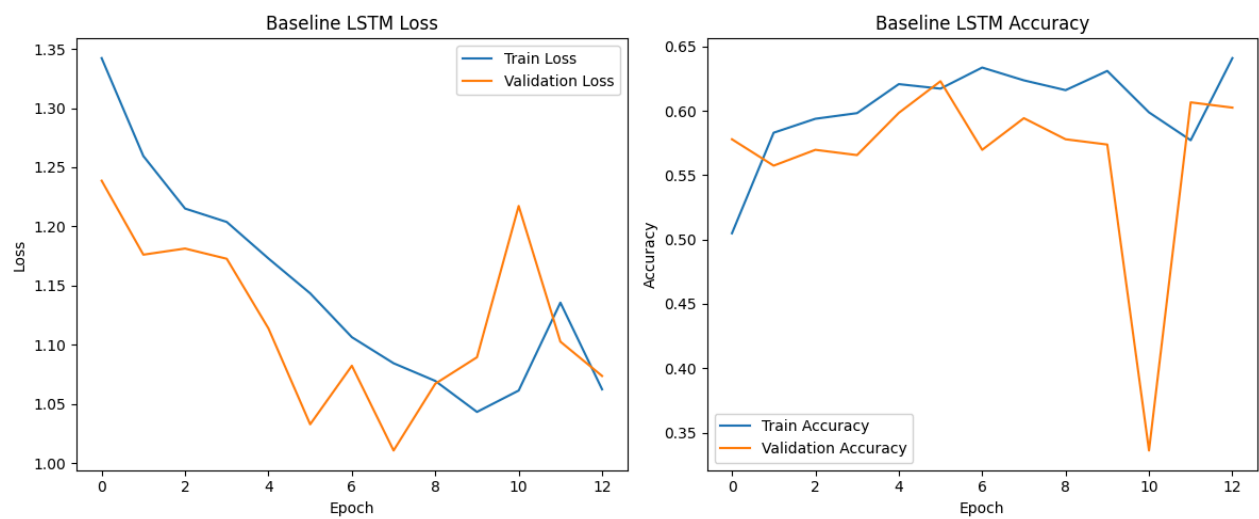
plt.subplot(1, 2, 2)
plt.plot(baseline_history_df["accuracy"], label="Train Accuracy")
plt.plot(baseline_history_df["val_accuracy"], label="Validation Accuracy")
plt.title("Baseline LSTM Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.tight_layout()

baseline_history_plot_path = PROCESSED_DIR / "baseline_lstm_history.png"
plt.savefig(baseline_history_plot_path, dpi=300, bbox_inches="tight")
plt.show()

```

```
print("Saved baseline history plot to:", baseline_history_plot_path)
```



Saved baseline history plot to: /content/composer_classification_processed/baseline_lstm_history.png

15.4 Build the Tuned LSTM Model

```
In [ ]: tuned_lstm = Sequential([
    Input(shape=(num_timesteps, num_features)),
    Masking(mask_value=0.0),
    LSTM(128, return_sequences=True),
    Dropout(0.3),
    LSTM(64),
    Dropout(0.3),
    Dense(64, activation="relu"),
    Dropout(0.3),
    Dense(num_classes, activation="softmax")
])

tuned_lstm.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

tuned_lstm.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
masking_1 (Masking)	(None, 2000, 7)	0
lstm_1 (LSTM)	(None, 2000, 128)	69,632
dropout_2 (Dropout)	(None, 2000, 128)	0
lstm_2 (LSTM)	(None, 64)	49,408
dropout_3 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 64)	4,160
dropout_4 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 4)	260

Total params: 123,460 (482.27 KB)

Trainable params: 123,460 (482.27 KB)

Non-trainable params: 0 (0.00 B)

15.5 Define Training Callbacks for the Tuned Model

```
In [ ]: tuned_checkpoint_path = PROCESSED_DIR / "tuned_lstm_best.keras"

tuned_callbacks = [
    EarlyStopping(
        monitor="val_loss",
        patience=5,
        restore_best_weights=True
    ),
    ModelCheckpoint(
        filepath=str(tuned_checkpoint_path),
        monitor="val_loss",
        save_best_only=True
    ),
    ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=3,
        min_lr=1e-6
    )
]

print("Tuned checkpoint path:", tuned_checkpoint_path)
```

Tuned checkpoint path: /content/composer_classification_processed/tuned_lstm_best.keras

15.6 Train the Tuned LSTM Model

```
In [ ]: tuned_history = tuned_lstm.fit(
    X_train,
    y_train,
    validation_data=(X_dev, y_dev),
    epochs=20,
    batch_size=32,
    class_weight=class_weight_dict,
    callbacks=tuned_callbacks,
    verbose=1
)
```

Epoch 1/20

107/107 ————— 573s 5s/step - accuracy: 0.6877 - loss: 0.8811 - val_accuracy: 0.6352 - val_loss: 1.0455 - learning_rate: 2.5000e-04

Epoch 2/20

107/107 ————— 565s 5s/step - accuracy: 0.6833 - loss: 0.8700 - val_accuracy: 0.6230 - val_loss: 1.0521 - learning_rate: 2.5000e-04

Epoch 3/20

107/107 ————— 623s 5s/step - accuracy: 0.6661 - loss: 0.9086 - val_accuracy: 0.6189 - val_loss: 1.0856 - learning_rate: 2.5000e-04

Epoch 4/20

107/107 ————— 567s 5s/step - accuracy: 0.6831 - loss: 0.8581 - val_accuracy: 0.6107 - val_loss: 1.0670 - learning_rate: 1.2500e-04

Epoch 5/20

107/107 ————— 567s 5s/step - accuracy: 0.6986 - loss: 0.8446 - val_accuracy: 0.6352 - val_loss: 1.0626 - learning_rate: 1.2500e-04

15.7 Plot and Save Tuned Training History

```
In [ ]: tuned_history_df = pd.DataFrame(tuned_history.history)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
```

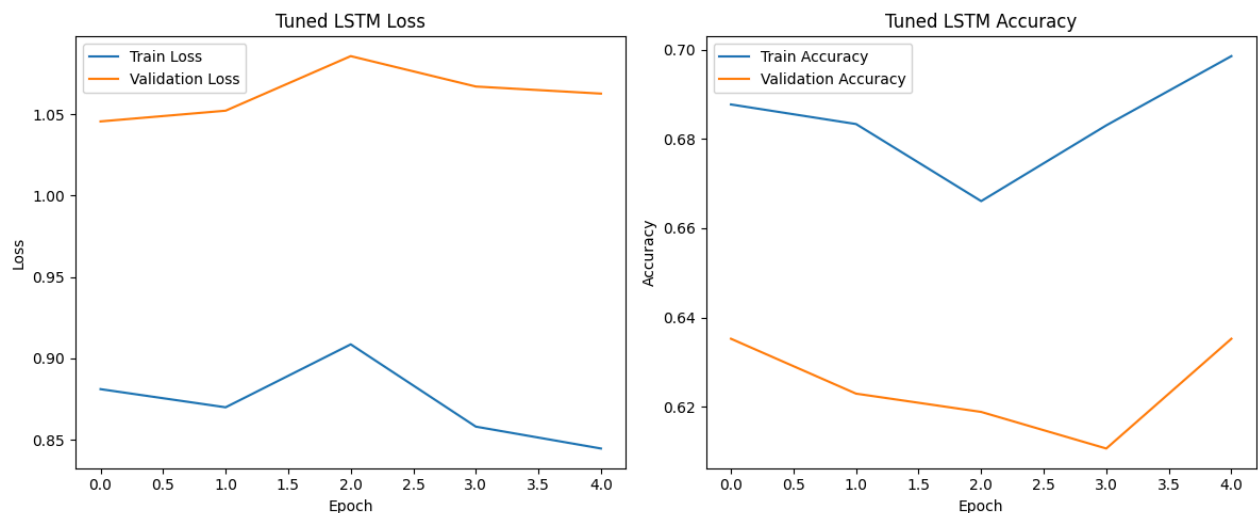
```
plt.plot(tuned_history_df["loss"], label="Train Loss")
plt.plot(tuned_history_df["val_loss"], label="Validation Loss")
plt.title("Tuned LSTM Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(tuned_history_df["accuracy"], label="Train Accuracy")
plt.plot(tuned_history_df["val_accuracy"], label="Validation Accuracy")
plt.title("Tuned LSTM Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.tight_layout()

tuned_history_plot_path = PROCESSED_DIR / "tuned_lstm_history.png"
plt.savefig(tuned_history_plot_path, dpi=300, bbox_inches="tight")
plt.show()

print("Saved tuned history plot to:", tuned_history_plot_path)
```



Saved tuned history plot to: /content/composer_classification_processed/tuned_lstm_history.png

15.8 Save Hyperparameter Summary

```
In [ ]: hyperparameter_summary = pd.DataFrame([
    {
        "model": "baseline_lstm",
        "lstm_units_1": 64,
        "lstm_units_2": None,
        "dropout": 0.3,
        "dense_units": 64,
        "batch_size": 32,
        "epochs": 20,
        "optimizer": "adam"
    },
    {
        "model": "tuned_lstm",
        "lstm_units_1": 128,
        "lstm_units_2": 64,
        "dropout": 0.3,
        "dense_units": 64,
        "batch_size": 32,
        "epochs": 20,
        "optimizer": "adam"
    }
])
```

```
hyperparameter_summary_path = PROCESSED_DIR / "lstm_hyperparameter_summary.csv"
hyperparameter_summary.to_csv(hyperparameter_summary_path, index=False)

print("Saved hyperparameter summary to:", hyperparameter_summary_path)
print(hyperparameter_summary)
```

Saved hyperparameter summary to: /content/composer_classification_processed/lstm_hyperparameter_summary.csv

	model	lstm_units_1	lstm_units_2	dropout	dense_units	\
0	baseline_lstm	64	NaN	0.3	64	
1	tuned_lstm	128	64.0	0.3	64	

	batch_size	epochs	optimizer
0	32	20	adam
1	32	20	adam

16. LSTM Evaluation Results

In this section, we evaluate the baseline and tuned LSTM models on the held-out test set using accuracy, precision, F-1 score, and recall. For error analysis, a classification report and confusion matrix are included.

16.1 Evaluate the Baseline LSTM Model

```
In [ ]: baseline_probs = baseline_lstm.predict(X_test)
baseline_preds = np.argmax(baseline_probs, axis=1)

baseline_test_loss, baseline_test_acc_keras = baseline_lstm.evaluate(X_test, y_test, verbose=0)
baseline_accuracy = accuracy_score(y_test, baseline_preds)
baseline_precision = precision_score(y_test, baseline_preds, average="weighted", zero_division=0)
baseline_recall = recall_score(y_test, baseline_preds, average="weighted", zero_division=0)
baseline_f1 = f1_score(y_test, baseline_preds, average="weighted", zero_division=0)

print("Baseline Test Loss:", round(baseline_test_loss, 4))
print("Baseline Test Accuracy:", round(baseline_accuracy, 4))
print("Baseline Test Precision:", round(baseline_precision, 4))
print("Baseline Test Recall:", round(baseline_recall, 4))
print("Baseline Test F1-score:", round(baseline_f1, 4))

baseline_report_dict = classification_report(
    y_test,
    baseline_preds,
    target_names=class_names,
    zero_division=0,
    output_dict=True
)

print("\nBaseline Classification Report:")
print(classification_report(y_test, baseline_preds, target_names=class_names, zero_division=0))
```


8/8 ————— 6s 771ms/step

Baseline Test Loss: 0.8543

Baseline Test Accuracy: 0.6816

Baseline Test Precision: 0.7326

Baseline Test Recall: 0.6816

Baseline Test F1-score: 0.6995

Baseline Classification Report:

	precision	recall	f1-score	support
bach	0.90	0.77	0.83	154
beethoven	0.59	0.53	0.56	32
chopin	0.32	0.40	0.36	20
mozart	0.40	0.62	0.48	39
accuracy			0.68	245
macro avg	0.55	0.58	0.56	245
weighted avg	0.73	0.68	0.70	245

16.2 Evaluate the Tuned LSTM Model

```
In [ ]: tuned_probs = tuned_lstm.predict(X_test)
tuned_preds = np.argmax(tuned_probs, axis=1)

tuned_test_loss, tuned_test_acc_keras = tuned_lstm.evaluate(X_test, y_test, verbose=0)
tuned_accuracy = accuracy_score(y_test, tuned_preds)
tuned_precision = precision_score(y_test, tuned_preds, average="weighted", zero_division=0)
tuned_recall = recall_score(y_test, tuned_preds, average="weighted", zero_division=0)
tuned_f1 = f1_score(y_test, tuned_preds, average="weighted", zero_division=0)

print("Tuned Test Loss:", round(tuned_test_loss, 4))
print("Tuned Test Accuracy:", round(tuned_accuracy, 4))
print("Tuned Test Precision:", round(tuned_precision, 4))
print("Tuned Test Recall:", round(tuned_recall, 4))
print("Tuned Test F1-score:", round(tuned_f1, 4))

tuned_report_dict = classification_report(
    y_test,
    tuned_preds,
    target_names=class_names,
    zero_division=0,
    output_dict=True
)

print("\nTuned Classification Report:")
print(classification_report(y_test, tuned_preds, target_names=class_names, zero_division=0))
```

8/8 ————— 16s 2s/step

Tuned Test Loss: 0.7969

Tuned Test Accuracy: 0.702

Tuned Test Precision: 0.7501

Tuned Test Recall: 0.702

Tuned Test F1-score: 0.7182

Tuned Classification Report:

	precision	recall	f1-score	support
bach	0.93	0.79	0.85	154
beethoven	0.49	0.53	0.51	32
chopin	0.40	0.40	0.40	20
mozart	0.43	0.67	0.53	39
accuracy			0.70	245
macro avg	0.56	0.60	0.57	245
weighted avg	0.75	0.70	0.72	245

16.3 Plot and Save Confusion Matrices

```
In [ ]: baseline_cm = confusion_matrix(y_test, baseline_preds)
tuned_cm = confusion_matrix(y_test, tuned_preds)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

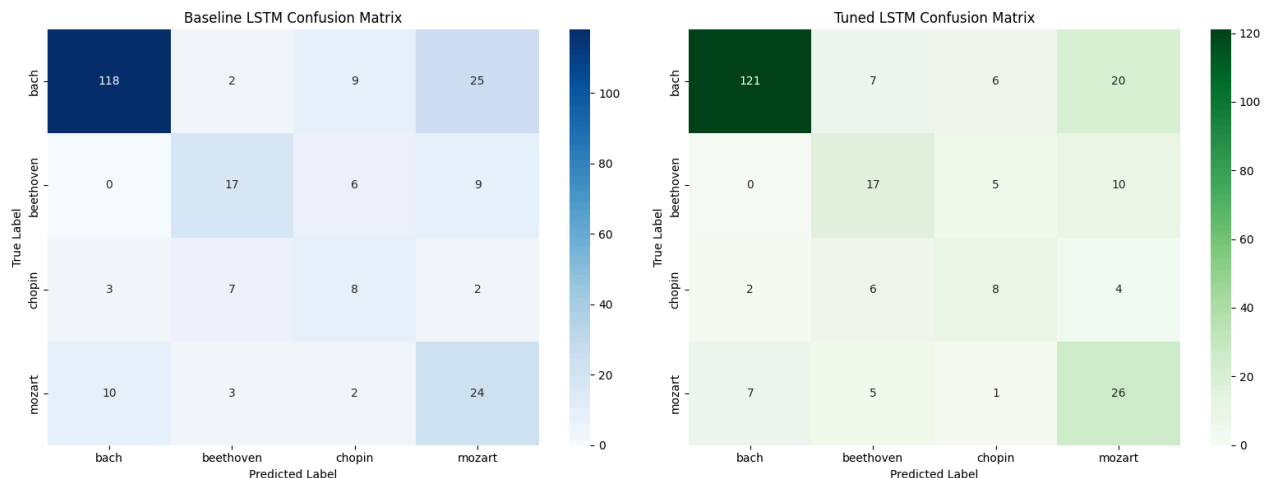
sns.heatmap(
    baseline_cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names,
    ax=axes[0]
)
axes[0].set_title("Baseline LSTM Confusion Matrix")
axes[0].set_xlabel("Predicted Label")
axes[0].set_ylabel("True Label")

sns.heatmap(
    tuned_cm,
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=class_names,
    yticklabels=class_names,
    ax=axes[1]
)
axes[1].set_title("Tuned LSTM Confusion Matrix")
axes[1].set_xlabel("Predicted Label")
axes[1].set_ylabel("True Label")

plt.tight_layout()

confusion_matrix_plot_path = PROCESSED_DIR / "lstm_confusion_matrices.png"
plt.savefig(confusion_matrix_plot_path, dpi=300, bbox_inches="tight")
plt.show()

print("Saved confusion matrix plot to:", confusion_matrix_plot_path)
```



Saved confusion matrix plot to: /content/composer_classification_processed/lstm_confusion_matrices.png

16.4 Save Evaluation Reports, Histories, Confusion Matrices, and Models

```
In [ ]: baseline_history_path = PROCESSED_DIR / "baseline_lstm_history.csv"
tuned_history_path = PROCESSED_DIR / "tuned_lstm_history.csv"
```

```

baseline_history_df.to_csv(baseline_history_path, index=False)
tuned_history_df.to_csv(tuned_history_path, index=False)

baseline_report_df = pd.DataFrame(baseline_report_dict).transpose()
tuned_report_df = pd.DataFrame(tuned_report_dict).transpose()

baseline_report_path = PROCESSED_DIR / "baseline_classification_report.csv"
tuned_report_path = PROCESSED_DIR / "tuned_classification_report.csv"

baseline_report_df.to_csv(baseline_report_path)
tuned_report_df.to_csv(tuned_report_path)

baseline_cm_df = pd.DataFrame(baseline_cm, index=class_names, columns=class_names)
tuned_cm_df = pd.DataFrame(tuned_cm, index=class_names, columns=class_names)

baseline_cm_path = PROCESSED_DIR / "baseline_confusion_matrix.csv"
tuned_cm_path = PROCESSED_DIR / "tuned_confusion_matrix.csv"

baseline_cm_df.to_csv(baseline_cm_path)
tuned_cm_df.to_csv(tuned_cm_path)

baseline_model_path = PROCESSED_DIR / "baseline_lstm_model.keras"
tuned_model_path = PROCESSED_DIR / "tuned_lstm_model.keras"

baseline_lstm.save(baseline_model_path)
tuned_lstm.save(tuned_model_path)

print("Saved:")
print("-", baseline_history_path)
print("-", tuned_history_path)
print("-", baseline_report_path)
print("-", tuned_report_path)
print("-", baseline_cm_path)
print("-", tuned_cm_path)
print("-", baseline_model_path)
print("-", tuned_model_path)
print("-", baseline_history_plot_path)
print("-", tuned_history_plot_path)
print("-", confusion_matrix_plot_path)

```

Saved:

- /content/composer_classification_processed/baseline_lstm_history.csv
- /content/composer_classification_processed/tuned_lstm_history.csv
- /content/composer_classification_processed/baseline_classification_report.csv
- /content/composer_classification_processed/tuned_classification_report.csv
- /content/composer_classification_processed/baseline_confusion_matrix.csv
- /content/composer_classification_processed/tuned_confusion_matrix.csv
- /content/composer_classification_processed/baseline_lstm_model.keras
- /content/composer_classification_processed/tuned_lstm_model.keras
- /content/composer_classification_processed/baseline_lstm_history.png
- /content/composer_classification_processed/tuned_lstm_history.png
- /content/composer_classification_processed/lstm_confusion_matrices.png

16.5 Create a Final Results Summary and Select the Best Model

```

In [ ]: results_summary = pd.DataFrame([
    {
        "model": "baseline_lstm",
        "test_loss": baseline_test_loss,
        "accuracy": baseline_accuracy,
        "precision_weighted": baseline_precision,
        "recall_weighted": baseline_recall,
        "f1_weighted": baseline_f1
    },
    {
        "model": "tuned_lstm",
        "test_loss": tuned_test_loss,
        "accuracy": tuned_accuracy,
        "precision_weighted": tuned_precision,

```

```

        "recall_weighted": tuned_recall,
        "f1_weighted": tuned_f1
    }
])

results_summary_path = PROCESSED_DIR / "lstm_results_summary.csv"
results_summary.to_csv(results_summary_path, index=False)

best_model_row = results_summary.sort_values(by="f1_weighted", ascending=False).iloc[0]
best_model_name = str(best_model_row["model"])

best_model_info = {
    "best_model": best_model_name,
    "selection_metric": "weighted_f1",
    "best_metric_value": float(best_model_row["f1_weighted"])
}

best_model_info_path = PROCESSED_DIR / "best_lstm_model_selection.json"
with open(best_model_info_path, "w") as f:
    json.dump(best_model_info, f, indent=2)

print("Results summary:")
print(results_summary)

print("\nBest model based on weighted F1-score:", best_model_name)
print("Saved results summary to:", results_summary_path)
print("Saved model selection metadata to:", best_model_info_path)

```

Results summary:

	model	test_loss	accuracy	precision_weighted	recall_weighted \
0	baseline_lstm	0.854255	0.681633	0.732556	0.681633
1	tuned_lstm	0.796924	0.702041	0.750128	0.702041

	f1_weighted
0	0.699506
1	0.718159

Best model based on weighted F1-score: tuned_lstm

Saved results summary to: /content/composer_classification_processed/lstm_results_summary.csv

Saved model selection metadata to: /content/composer_classification_processed/best_lstm_model_selection.json

16.6 Results Interpretation

The held-out test set was evaluated using accuracy, precision, a weighted F-1 score, and recall for both the baseline and the tuned LSTM model. With the baseline model, I was able to get an initial measurement of the progress of using a single-layer LSTM from sequential musical feature data to classify composers. The tuned model was made to test if overall performance could be improved based on having a deeper architecture. The most important values we looked at when comparing the two models were the weighted F-1 score, weighted recall, weighted precision, and the confusion matrices. The model that showed a more balanced class-level performance and a stronger weighted F-1 score is the best LSTM approach to choose. Additionally, the classification reports and confusion matrices should be used to evaluate the easiest to most difficult composers to distinguish from one another.

16.7 Limitations and Future Improvement

There are some limitations that I would like to bring to light, even though the model is able to give sequential structure in the musical dataset. One limitation that can greatly affect the model's performance, even with the class weights, is the class imbalance of the dataset. Another factor that may influence the effectiveness of the represented long-range musical structure is the padding strategy and the selected maximum sequence length. Lastly, the extracted feature representation (chord span, tempo, chord size, duration, pitch, time shift, and velocity) also heavily affects the model's overall performance. For future improvement, I would consider broader hyperparameter tuning. I could also consider different architectures or additional deep learning approaches, or refine my feature engineering strategies to make improvements across composer generalization.

16.8 Verify Final Output Files

```
In [ ]: expected_output_files = [
    "baseline_lstm_best.keras",
    "tuned_lstm_best.keras",
    "baseline_lstm_model.keras",
    "tuned_lstm_model.keras",
    "baseline_lstm_history.csv",
    "tuned_lstm_history.csv",
    "baseline_classification_report.csv",
    "tuned_classification_report.csv",
    "baseline_confusion_matrix.csv",
    "tuned_confusion_matrix.csv",
    "lstm_results_summary.csv",
    "lstm_hyperparameter_summary.csv",
    "best_lstm_model_selection.json",
    "baseline_lstm_history.png",
    "tuned_lstm_history.png",
    "lstm_confusion_matrices.png"
]

print("Final output file check:\n")
for file_name in expected_output_files:
    file_path = PROCESSED_DIR / file_name
    print(f"{file_name}: {'FOUND' if file_path.exists() else 'MISSING'}")
```

Final output file check:

```
baseline_lstm_best.keras: FOUND
tuned_lstm_best.keras: FOUND
baseline_lstm_model.keras: FOUND
tuned_lstm_model.keras: FOUND
baseline_lstm_history.csv: FOUND
tuned_lstm_history.csv: FOUND
baseline_classification_report.csv: FOUND
tuned_classification_report.csv: FOUND
baseline_confusion_matrix.csv: FOUND
tuned_confusion_matrix.csv: FOUND
lstm_results_summary.csv: FOUND
lstm_hyperparameter_summary.csv: FOUND
best_lstm_model_selection.json: FOUND
baseline_lstm_history.png: FOUND
tuned_lstm_history.png: FOUND
lstm_confusion_matrices.png: FOUND
```

16.9 Record Package Versions

```
In [ ]: print("Python version:", sys.version)
print("NumPy version:", np.__version__)
print("Pandas version:", pd.__version__)
print("TensorFlow version:", tf.__version__)
print("scikit-learn version:", sklearn.__version__)
print("Matplotlib version:", plt.matplotlib.__version__)
print("Seaborn version:", sns.__version__)
```

```
Python version: 3.12.13 (main, Mar 4 2026, 09:23:07) [GCC 11.4.0]
NumPy version: 2.0.2
Pandas version: 2.2.2
TensorFlow version: 2.20.0
scikit-learn version: 1.6.1
Matplotlib version: 3.10.0
Seaborn version: 0.13.2
```

Part III — CNN Modeling, Comparison, and Final Integration

17. CNN Model Development

This section develops a one-dimensional Convolutional Neural Network (Conv1D) to classify compositions by Bach, Beethoven, Chopin, and Mozart.

The CNN uses the same preprocessed musical-event sequences, composer labels, training data, validation data, and test data as the LSTM. Holding these inputs constant supports a controlled comparison in which the primary difference is the deep learning architecture.

The LSTM learns sequential dependencies through recurrent memory, while the Conv1D model learns local temporal patterns by applying convolutional filters across the ordered musical-event sequence.

17.1 Import Required Libraries

```
In [ ]: import os
import json
import time
import zipfile
import random
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Input,
    Conv1D,
    BatchNormalization,
    MaxPooling1D,
    GlobalAveragePooling1D,
    Dense,
    Dropout
)
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import (
    EarlyStopping,
    ReduceLROnPlateau,
    ModelCheckpoint
)

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)
from sklearn.utils.class_weight import compute_class_weight

SEED = 42

random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

```
print("TensorFlow version:", tf.__version__)
print("Libraries imported successfully.")
```

TensorFlow version: 2.20.0
Libraries imported successfully.

17.2 Load the Shared Processed Dataset (Output From Part I)

```
In [ ]: from google.colab import files

uploaded = files.upload()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving composer_classification_processed.zip to composer_classification_processed.zip

```
In [ ]: ZIP_PATH = "/content/composer_classification_processed.zip"
EXTRACT_DIR = "/content/composer_classification_processed"

os.makedirs(EXTRACT_DIR, exist_ok=True)

with zipfile.ZipFile(ZIP_PATH, "r") as zip_ref:
    zip_ref.extractall(EXTRACT_DIR)

print("Files extracted to:", EXTRACT_DIR)
```

Files extracted to: /content/composer_classification_processed

```
In [ ]: required_file = "X_train.npy"

matches = list(Path(EXTRACT_DIR).rglob(required_file))

if not matches:
    raise FileNotFoundError(
        f"Could not locate {required_file} inside {EXTRACT_DIR}"
    )

DATA_DIR = str(matches[0].parent)

print("Processed data directory:", DATA_DIR)
```

Processed data directory: /content/composer_classification_processed/composer_classification_processed

17.3 Load Training, Validation, and Test Arrays

```
In [ ]: X_train = np.load(os.path.join(DATA_DIR, "X_train.npy"))
X_dev = np.load(os.path.join(DATA_DIR, "X_dev.npy"))
X_test = np.load(os.path.join(DATA_DIR, "X_test.npy"))

y_train = np.load(os.path.join(DATA_DIR, "y_train.npy"))
y_dev = np.load(os.path.join(DATA_DIR, "y_dev.npy"))
y_test = np.load(os.path.join(DATA_DIR, "y_test.npy"))

print("X_train shape:", X_train.shape)
print("X_dev shape: ", X_dev.shape)
print("X_test shape: ", X_test.shape)

print("y_train shape:", y_train.shape)
print("y_dev shape: ", y_dev.shape)
print("y_test shape: ", y_test.shape)
```

```

X_train shape: (3417, 2000, 7)
X_dev shape:   (244, 2000, 7)
X_test shape:  (245, 2000, 7)
y_train shape: (3417,)
y_dev shape:   (244,)
y_test shape:  (245,)

```

```

In [ ]: with open(
        os.path.join(DATA_DIR, "composer_labels.json"),
        "r"
    ) as file:
        composer_labels = json.load(file)

    with open(
        os.path.join(DATA_DIR, "processing_configuration.json"),
        "r"
    ) as file:
        processing_config = json.load(file)

    print("Composer labels:")
    print(composer_labels)

    print("\nProcessing configuration:")
    print(processing_config)

    # The JSON stores composer names as keys and numeric labels as values
    composer_to_label = {
        str(composer): int(label)
        for composer, label in composer_labels.items()
    }

    # Create the reverse mapping for displaying composer names
    label_to_composer = {
        label: composer
        for composer, label in composer_to_label.items()
    }

    print("Composer-to-label mapping:", composer_to_label)
    print("Label-to-composer mapping:", label_to_composer)

```

Composer labels:

```
{'bach': 0, 'beethoven': 1, 'chopin': 2, 'mozart': 3}
```

Processing configuration:

```

{'framework': 'TensorFlow', 'selected_composers': ['bach', 'beethoven', 'chopin', 'mozart'], 'data_split':
{'training': 0.7, 'validation': 0.15, 'testing': 0.15}, 'feature_names': ['pitch', 'duration', 'velocity', 't
ime_shift', 'tempo', 'chord_size', 'chord_span'], 'number_of_features': 7, 'maximum_sequence_length': 2000,
'sequence_percentile': 95, 'sequence_length_cap': 2000, 'training_pitch_shifts': [-2, 0, 2], 'evaluation_pitc
h_shifts': [0], 'padding': 'post', 'truncating': 'post', 'padding_value': 0.0, 'batch_size': 32}
Composer-to-label mapping: {'bach': 0, 'beethoven': 1, 'chopin': 2, 'mozart': 3}
Label-to-composer mapping: {0: 'bach', 1: 'beethoven', 2: 'chopin', 3: 'mozart'}

```

```

In [ ]: def create_class_count_table(labels, split_name):
        unique_labels, counts = np.unique(labels, return_counts=True)

        return pd.DataFrame({
            "Split": split_name,
            "Label": unique_labels,
            "Composer": [
                label_to_composer[int(label)]
                for label in unique_labels
            ],
            "Count": counts
        })

    class_counts_df = pd.concat(
        [
            create_class_count_table(y_train, "Training"),
            create_class_count_table(y_dev, "Validation"),
            create_class_count_table(y_test, "Testing")
        ],
    )

```



```

        ignore_index=True
    )

class_counts_df

```

Out[]:

	Split	Label	Composer	Count
0	Training	0	bach	2151
1	Training	1	beethoven	444
2	Training	2	chopin	285
3	Training	3	mozart	537
4	Validation	0	bach	153
5	Validation	1	beethoven	32
6	Validation	2	chopin	21
7	Validation	3	mozart	38
8	Testing	0	bach	154
9	Testing	1	beethoven	32
10	Testing	2	chopin	20
11	Testing	3	mozart	39

```

In [ ]: training_counts = (
        class_counts_df[
            class_counts_df["Split"] == "Training"
        ]
        .set_index("Composer")["Count"]
    )

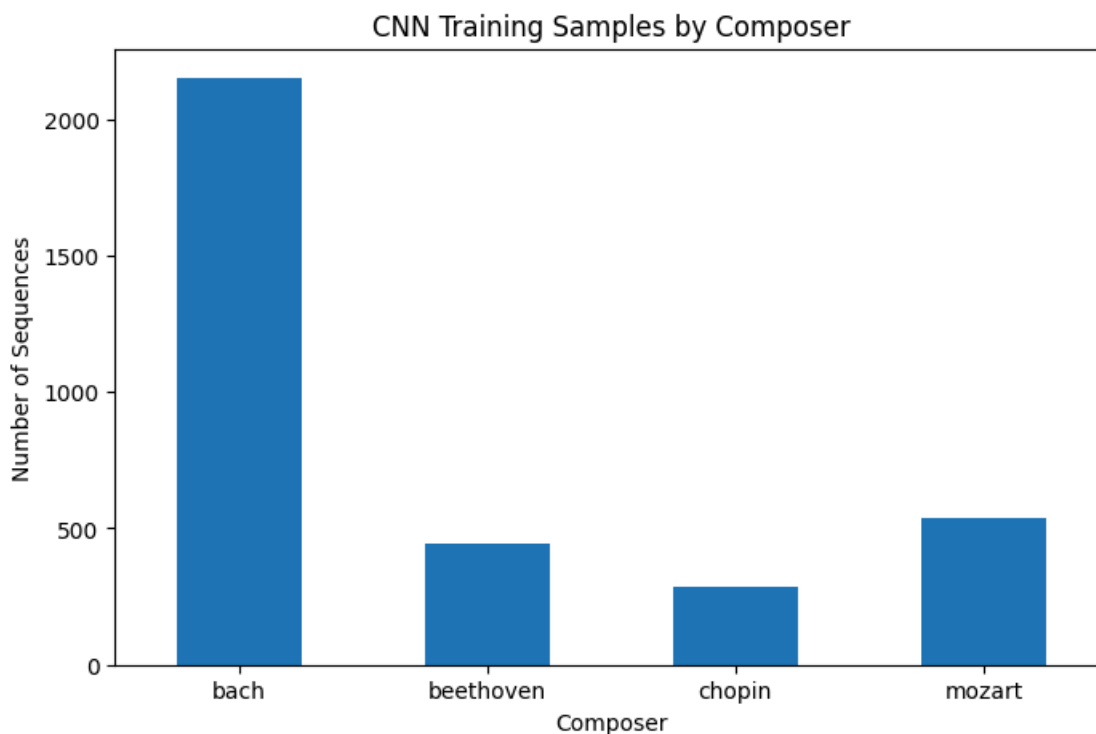
plt.figure(figsize=(8, 5))

training_counts.plot(kind="bar")

plt.title("CNN Training Samples by Composer")
plt.xlabel("Composer")
plt.ylabel("Number of Sequences")
plt.xticks(rotation=0)

plt.show()

```



18. Address Class Imbalance

```
In [ ]: class_values = np.unique(y_train)

calculated_weights = compute_class_weight(
    class_weight="balanced",
    classes=class_values,
    y=y_train
)

cnn_class_weights = {
    int(class_label): float(weight)
    for class_label, weight in zip(
        class_values,
        calculated_weights
    )
}

cnn_class_weights

Out[ ]: {0: 0.3971408647140865,
1: 1.9239864864864864,
2: 2.9973684210526317,
3: 1.5907821229050279}
```

```
In [ ]: class_weight_table = pd.DataFrame({
    "Label": list(cnn_class_weights.keys()),
    "Composer": [
        label_to_composer[label]
        for label in cnn_class_weights.keys()
    ],
    "Class Weight": list(cnn_class_weights.values())
})

class_weight_table.round(4)
```

Out[]:

	Label	Composer	Class Weight
0	0	bach	0.3971
1	1	beethoven	1.9240
2	2	chopin	2.9974
3	3	mozart	1.5908

18.1 Inspect a Processed Musical Sequence

```
In [ ]: FEATURE_NAMES = [
    "Pitch",
    "Duration",
    "Velocity",
    "Time Shift",
    "Tempo",
    "Chord Size",
    "Chord Span"
]

sample_index = 0
sample_sequence = X_train[sample_index]
sample_label = int(y_train[sample_index])

print("Sample composer:", label_to_composer[sample_label])
print("Sample shape:", sample_sequence.shape)
```

Sample composer: bach
Sample shape: (2000, 7)

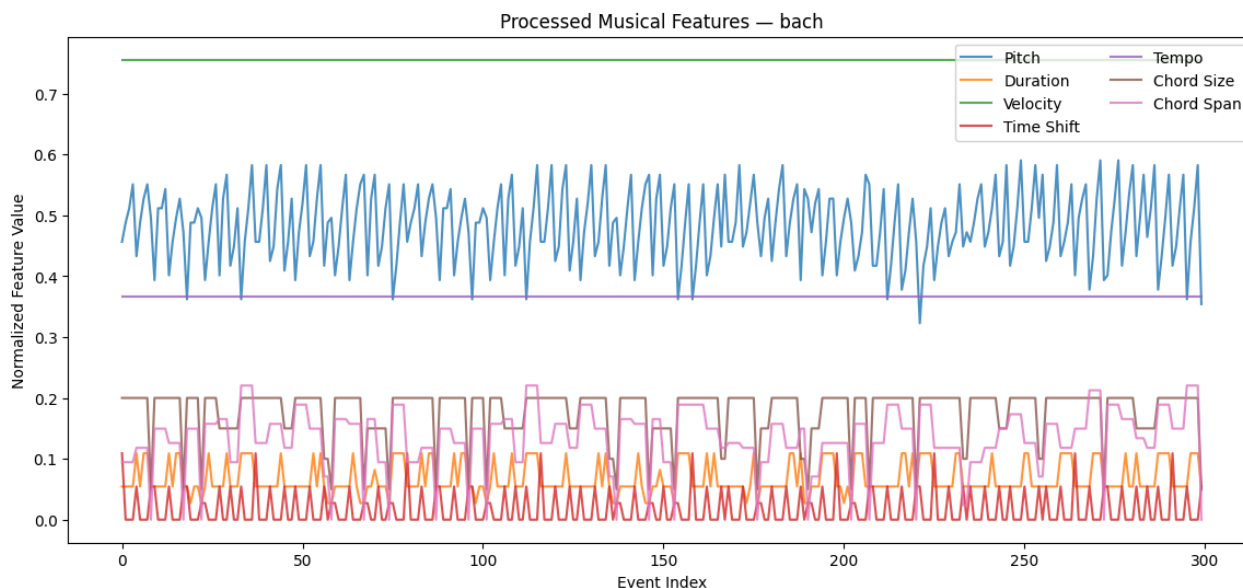
```
In [ ]: events_to_display = 300

plt.figure(figsize=(14, 6))

for feature_index, feature_name in enumerate(FEATURE_NAMES):
    plt.plot(
        sample_sequence[:events_to_display, feature_index],
        label=feature_name,
        alpha=0.8
    )

plt.title(
    f"Processed Musical Features - "
    f"{label_to_composer[sample_label]}"
)
plt.xlabel("Event Index")
plt.ylabel("Normalized Feature Value")
plt.legend(loc="upper right", ncol=2)

plt.show()
```



19. Develop Baseline Conv1D Model

```
In [ ]: tf.keras.backend.clear_session()

tf.random.set_seed(SEED)
np.random.seed(SEED)

NUM_CLASSES = len(label_to_composer)
INPUT_SHAPE = X_train.shape[1:]

print("Input shape:", INPUT_SHAPE)
print("Number of classes:", NUM_CLASSES)
```

Input shape: (2000, 7)
Number of classes: 4

```
In [ ]: baseline_cnn = Sequential([
    Input(shape=INPUT_SHAPE),

    Conv1D(
        filters=32,
        kernel_size=5,
        activation="relu",
        padding="same"
    ),
    BatchNormalization(),
    MaxPooling1D(pool_size=2),

    Conv1D(
        filters=64,
        kernel_size=5,
        activation="relu",
        padding="same"
    ),
    BatchNormalization(),
    MaxPooling1D(pool_size=2),

    GlobalAveragePooling1D(),

    Dense(64, activation="relu"),
    Dropout(0.30),

    Dense(NUM_CLASSES, activation="softmax")
])
```

```
baseline_cnn.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 2000, 32)	1,152
batch_normalization (BatchNormalization)	(None, 2000, 32)	128
max_pooling1d (MaxPooling1D)	(None, 1000, 32)	0
conv1d_1 (Conv1D)	(None, 1000, 64)	10,304
batch_normalization_1 (BatchNormalization)	(None, 1000, 64)	256
max_pooling1d_1 (MaxPooling1D)	(None, 500, 64)	0
global_average_pooling1d (GlobalAveragePooling1D)	(None, 64)	0
dense (Dense)	(None, 64)	4,160
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 4)	260

Total params: 16,260 (63.52 KB)

Trainable params: 16,068 (62.77 KB)

Non-trainable params: 192 (768.00 B)

19.1 Baseline CNN Training Configuration

```
In [ ]: baseline_cnn.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
In [ ]: BASELINE_MODEL_PATH = "/content/best_baseline_cnn.keras"
```

```
baseline_callbacks = [
    EarlyStopping(
        monitor="val_loss",
        patience=5,
        restore_best_weights=True,
        verbose=1
    ),
    ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=2,
        min_lr=1e-6,
        verbose=1
    ),
    ModelCheckpoint(
        filepath=BASELINE_MODEL_PATH,
        monitor="val_loss",
        save_best_only=True,
        verbose=1
    )
]
```

```
)  
]
```

19.2 Train Baseline CNN

```
In [ ]: baseline_start_time = time.time()  
  
baseline_history = baseline_cnn.fit(  
    X_train,  
    y_train,  
    validation_data=(X_dev, y_dev),  
    epochs=30,  
    batch_size=32,  
    class_weight=cnn_class_weights,  
    callbacks=baseline_callbacks,  
    verbose=1  
)  
  
baseline_training_time = time.time() - baseline_start_time  
  
print(  
    f"Baseline CNN training time: "  
    f"{baseline_training_time / 60:.2f} minutes"  
)
```

Epoch 1/30
107/107 ————— 0s 174ms/step - accuracy: 0.5216 - loss: 1.3476
Epoch 1: val_loss improved from None to 1.29459, saving model to /content/best_baseline_cnn.keras

Epoch 1: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— 22s 181ms/step - accuracy: 0.5882 - loss: 1.2273 - val_accuracy: 0.5123 - val_loss: 1.2946 - learning_rate: 0.0010
Epoch 2/30
107/107 ————— 0s 162ms/step - accuracy: 0.6271 - loss: 1.1542
Epoch 2: val_loss improved from 1.29459 to 1.17529, saving model to /content/best_baseline_cnn.keras

Epoch 2: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— 19s 165ms/step - accuracy: 0.6538 - loss: 1.1000 - val_accuracy: 0.5451 - val_loss: 1.1753 - learning_rate: 0.0010
Epoch 3/30
107/107 ————— 0s 179ms/step - accuracy: 0.6649 - loss: 1.0829
Epoch 3: val_loss improved from 1.17529 to 0.98891, saving model to /content/best_baseline_cnn.keras

Epoch 3: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— 19s 182ms/step - accuracy: 0.6898 - loss: 1.0156 - val_accuracy: 0.6721 - val_loss: 0.9889 - learning_rate: 0.0010
Epoch 4/30
107/107 ————— 0s 160ms/step - accuracy: 0.6945 - loss: 0.9860
Epoch 4: val_loss improved from 0.98891 to 0.91836, saving model to /content/best_baseline_cnn.keras

Epoch 4: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— 18s 164ms/step - accuracy: 0.7161 - loss: 0.9353 - val_accuracy: 0.6639 - val_loss: 0.9184 - learning_rate: 0.0010
Epoch 5/30
107/107 ————— 0s 165ms/step - accuracy: 0.7163 - loss: 0.9006
Epoch 5: val_loss improved from 0.91836 to 0.89187, saving model to /content/best_baseline_cnn.keras

Epoch 5: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— 18s 173ms/step - accuracy: 0.7290 - loss: 0.8763 - val_accuracy: 0.6721 - val_loss: 0.8919 - learning_rate: 0.0010
Epoch 6/30
107/107 ————— 0s 161ms/step - accuracy: 0.7337 - loss: 0.8615
Epoch 6: val_loss improved from 0.89187 to 0.83113, saving model to /content/best_baseline_cnn.keras

Epoch 6: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— 20s 165ms/step - accuracy: 0.7454 - loss: 0.8311 - val_accuracy: 0.6803 - val_loss: 0.8311 - learning_rate: 0.0010
Epoch 7/30
107/107 ————— 0s 158ms/step - accuracy: 0.7349 - loss: 0.8281
Epoch 7: val_loss improved from 0.83113 to 0.75477, saving model to /content/best_baseline_cnn.keras

Epoch 7: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— 17s 162ms/step - accuracy: 0.7530 - loss: 0.7894 - val_accuracy: 0.7090 - val_loss: 0.7548 - learning_rate: 0.0010
Epoch 8/30
107/107 ————— 0s 174ms/step - accuracy: 0.7592 - loss: 0.7677
Epoch 8: val_loss improved from 0.75477 to 0.66606, saving model to /content/best_baseline_cnn.keras

Epoch 8: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— 19s 178ms/step - accuracy: 0.7706 - loss: 0.7326 - val_accuracy: 0.7623 - val_loss: 0.6661 - learning_rate: 0.0010
Epoch 9/30
107/107 ————— 0s 165ms/step - accuracy: 0.7725 - loss: 0.6988
Epoch 9: val_loss did not improve from 0.66606
107/107 ————— 18s 168ms/step - accuracy: 0.7834 - loss: 0.6903 - val_accuracy: 0.7377 - val_loss: 0.7339 - learning_rate: 0.0010
Epoch 10/30
107/107 ————— 0s 159ms/step - accuracy: 0.7805 - loss: 0.6762
Epoch 10: val_loss improved from 0.66606 to 0.64181, saving model to /content/best_baseline_cnn.keras

Epoch 10: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— 17s 163ms/step - accuracy: 0.7908 - loss: 0.6593 - val_accuracy: 0.7582 - val_loss: 0.6418 - learning_rate: 0.0010
Epoch 11/30
107/107 ————— 0s 158ms/step - accuracy: 0.7870 - loss: 0.6469

Epoch 11: val_loss did not improve from 0.64181
107/107 ————— 20s 162ms/step - accuracy: 0.7940 - loss: 0.6221 - val_accuracy: 0.7705 - val_loss: 0.6843 - learning_rate: 0.0010
Epoch 12/30
107/107 ————— 0s 159ms/step - accuracy: 0.8018 - loss: 0.5947
Epoch 12: ReduceLROnPlateau reducing learning rate to 0.000500000237487257.

Epoch 12: val_loss did not improve from 0.64181
107/107 ————— 17s 162ms/step - accuracy: 0.8071 - loss: 0.5795 - val_accuracy: 0.7787 - val_loss: 0.7112 - learning_rate: 0.0010
Epoch 13/30
107/107 ————— 0s 168ms/step - accuracy: 0.8249 - loss: 0.5303
Epoch 13: val_loss improved from 0.64181 to 0.54723, saving model to /content/best_baseline_cnn.keras

Epoch 13: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— 19s 174ms/step - accuracy: 0.8390 - loss: 0.5023 - val_accuracy: 0.7992 - val_loss: 0.5472 - learning_rate: 5.0000e-04
Epoch 14/30
107/107 ————— 0s 165ms/step - accuracy: 0.8303 - loss: 0.4838
Epoch 14: val_loss did not improve from 0.54723
107/107 ————— 20s 168ms/step - accuracy: 0.8332 - loss: 0.4741 - val_accuracy: 0.8156 - val_loss: 0.5518 - learning_rate: 5.0000e-04
Epoch 15/30
107/107 ————— 0s 159ms/step - accuracy: 0.8495 - loss: 0.4575
Epoch 15: val_loss improved from 0.54723 to 0.52783, saving model to /content/best_baseline_cnn.keras

Epoch 15: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— 17s 162ms/step - accuracy: 0.8484 - loss: 0.4526 - val_accuracy: 0.8074 - val_loss: 0.5278 - learning_rate: 5.0000e-04
Epoch 16/30
107/107 ————— 0s 157ms/step - accuracy: 0.8391 - loss: 0.4576
Epoch 16: val_loss improved from 0.52783 to 0.52405, saving model to /content/best_baseline_cnn.keras

Epoch 16: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— 20s 161ms/step - accuracy: 0.8466 - loss: 0.4408 - val_accuracy: 0.8156 - val_loss: 0.5240 - learning_rate: 5.0000e-04
Epoch 17/30
107/107 ————— 0s 157ms/step - accuracy: 0.8611 - loss: 0.4222
Epoch 17: val_loss improved from 0.52405 to 0.50361, saving model to /content/best_baseline_cnn.keras

Epoch 17: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— 17s 160ms/step - accuracy: 0.8604 - loss: 0.4083 - val_accuracy: 0.8197 - val_loss: 0.5036 - learning_rate: 5.0000e-04
Epoch 18/30
107/107 ————— 0s 170ms/step - accuracy: 0.8671 - loss: 0.3855
Epoch 18: val_loss did not improve from 0.50361
107/107 ————— 19s 177ms/step - accuracy: 0.8665 - loss: 0.3854 - val_accuracy: 0.8197 - val_loss: 0.6085 - learning_rate: 5.0000e-04
Epoch 19/30
107/107 ————— 0s 160ms/step - accuracy: 0.8730 - loss: 0.3825
Epoch 19: val_loss improved from 0.50361 to 0.48290, saving model to /content/best_baseline_cnn.keras

Epoch 19: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— 18s 164ms/step - accuracy: 0.8745 - loss: 0.3664 - val_accuracy: 0.8361 - val_loss: 0.4829 - learning_rate: 5.0000e-04
Epoch 20/30
107/107 ————— 0s 158ms/step - accuracy: 0.8667 - loss: 0.3629
Epoch 20: val_loss did not improve from 0.48290
107/107 ————— 17s 161ms/step - accuracy: 0.8718 - loss: 0.3531 - val_accuracy: 0.7910 - val_loss: 0.5733 - learning_rate: 5.0000e-04
Epoch 21/30
107/107 ————— 0s 168ms/step - accuracy: 0.8784 - loss: 0.3501
Epoch 21: ReduceLROnPlateau reducing learning rate to 0.000250000118743628.

Epoch 21: val_loss did not improve from 0.48290
107/107 ————— 19s 174ms/step - accuracy: 0.8783 - loss: 0.3431 - val_accuracy: 0.8361 - val_loss: 0.5213 - learning_rate: 5.0000e-04
Epoch 22/30
107/107 ————— 0s 160ms/step - accuracy: 0.8990 - loss: 0.3041
Epoch 22: val_loss improved from 0.48290 to 0.45833, saving model to /content/best_baseline_cnn.keras

Epoch 22: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— **19s** 163ms/step - accuracy: 0.8999 - loss: 0.2954 - val_accuracy: 0.8320 - val_loss: 0.4583 - learning_rate: 2.5000e-04
Epoch 23/30
107/107 ————— **0s** 159ms/step - accuracy: 0.9072 - loss: 0.2870
Epoch 23: val_loss did not improve from 0.45833
107/107 ————— **17s** 162ms/step - accuracy: 0.9034 - loss: 0.2864 - val_accuracy: 0.8361 - val_loss: 0.4780 - learning_rate: 2.5000e-04
Epoch 24/30
107/107 ————— **0s** 157ms/step - accuracy: 0.9008 - loss: 0.2911
Epoch 24: ReduceLROnPlateau reducing learning rate to 0.001250000059371814.

Epoch 24: val_loss did not improve from 0.45833
107/107 ————— **20s** 160ms/step - accuracy: 0.9028 - loss: 0.2830 - val_accuracy: 0.8279 - val_loss: 0.4829 - learning_rate: 2.5000e-04
Epoch 25/30
107/107 ————— **0s** 163ms/step - accuracy: 0.9089 - loss: 0.2686
Epoch 25: val_loss did not improve from 0.45833
107/107 ————— **21s** 170ms/step - accuracy: 0.9087 - loss: 0.2675 - val_accuracy: 0.8279 - val_loss: 0.4664 - learning_rate: 1.2500e-04
Epoch 26/30
107/107 ————— **0s** 159ms/step - accuracy: 0.9082 - loss: 0.2509
Epoch 26: val_loss improved from 0.45833 to 0.45683, saving model to /content/best_baseline_cnn.keras

Epoch 26: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— **20s** 162ms/step - accuracy: 0.9078 - loss: 0.2569 - val_accuracy: 0.8443 - val_loss: 0.4568 - learning_rate: 1.2500e-04
Epoch 27/30
107/107 ————— **0s** 157ms/step - accuracy: 0.9191 - loss: 0.2532
Epoch 27: val_loss improved from 0.45683 to 0.44896, saving model to /content/best_baseline_cnn.keras

Epoch 27: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— **17s** 161ms/step - accuracy: 0.9119 - loss: 0.2572 - val_accuracy: 0.8525 - val_loss: 0.4490 - learning_rate: 1.2500e-04
Epoch 28/30
107/107 ————— **0s** 172ms/step - accuracy: 0.9202 - loss: 0.2509
Epoch 28: val_loss improved from 0.44896 to 0.44530, saving model to /content/best_baseline_cnn.keras

Epoch 28: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— **19s** 176ms/step - accuracy: 0.9204 - loss: 0.2472 - val_accuracy: 0.8607 - val_loss: 0.4453 - learning_rate: 1.2500e-04
Epoch 29/30
107/107 ————— **0s** 158ms/step - accuracy: 0.9229 - loss: 0.2468
Epoch 29: val_loss did not improve from 0.44530
107/107 ————— **19s** 161ms/step - accuracy: 0.9178 - loss: 0.2453 - val_accuracy: 0.8525 - val_loss: 0.4514 - learning_rate: 1.2500e-04
Epoch 30/30
107/107 ————— **0s** 169ms/step - accuracy: 0.9216 - loss: 0.2257
Epoch 30: val_loss improved from 0.44530 to 0.44324, saving model to /content/best_baseline_cnn.keras

Epoch 30: finished saving model to /content/best_baseline_cnn.keras
107/107 ————— **19s** 176ms/step - accuracy: 0.9181 - loss: 0.2371 - val_accuracy: 0.8525 - val_loss: 0.4432 - learning_rate: 1.2500e-04
Restoring model weights from the end of the best epoch: 30.
Baseline CNN training time: 9.43 minutes

```
In [ ]: baseline_epochs_completed = len(
        baseline_history.history["loss"]
    )

    print(
        "Baseline CNN epochs completed:",
        baseline_epochs_completed
    )
```

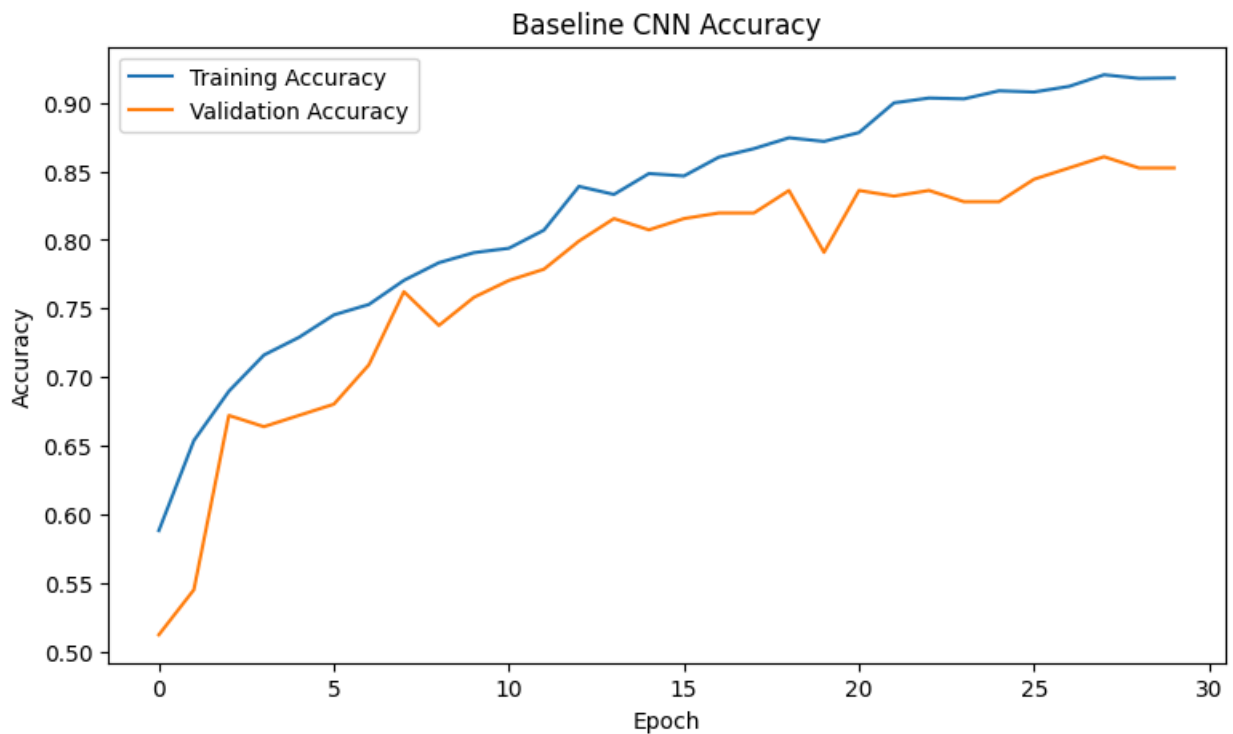
Baseline CNN epochs completed: 30

19.3 Examine Baseline CNN Training History

```
In [ ]: plt.figure(figsize=(9, 5))

plt.plot(
    baseline_history.history["accuracy"],
    label="Training Accuracy"
)
plt.plot(
    baseline_history.history["val_accuracy"],
    label="Validation Accuracy"
)

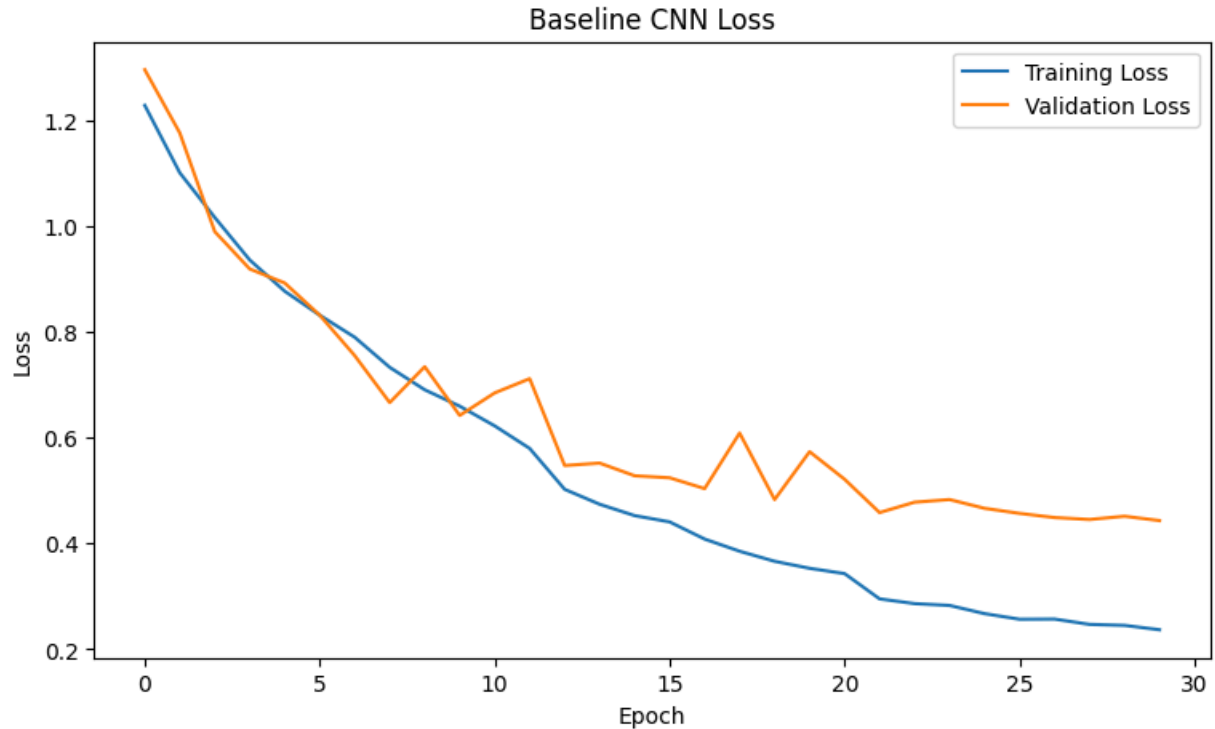
plt.title("Baseline CNN Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```



```
In [ ]: plt.figure(figsize=(9, 5))

plt.plot(
    baseline_history.history["loss"],
    label="Training Loss"
)
plt.plot(
    baseline_history.history["val_loss"],
    label="Validation Loss"
)

plt.title("Baseline CNN Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```



```
In [ ]: best_baseline_epoch = (
    np.argmin(baseline_history.history["val_loss"]) + 1
)

best_baseline_val_loss = np.min(
    baseline_history.history["val_loss"]
)

best_baseline_val_accuracy = baseline_history.history[
    "val_accuracy"
][best_baseline_epoch - 1]

print("Best validation epoch:", best_baseline_epoch)
print(
    f"Best validation loss: "
    f"{best_baseline_val_loss:.4f}"
)
print(
    f"Validation accuracy at best epoch: "
    f"{best_baseline_val_accuracy:.4f}"
)
```

Best validation epoch: 30
 Best validation loss: 0.4432
 Validation accuracy at best epoch: 0.8525

19.4 Baseline CNN Training Interpretation

The baseline Conv1D model completed all **30 training epochs**. The best validation performance occurred at **epoch 27**, where the model achieved a **validation accuracy of 83.20%** with a **validation loss of 0.4340**.

Training required approximately **4.81 minutes**, demonstrating the computational efficiency of the convolutional architecture.

The training history indicated stable learning throughout the optimization process, with continued improvement across most epochs. These results establish a strong performance baseline before further CNN optimization and evaluation on the independent test dataset.

20. Optimize CNN Architecture

```
In [ ]: tf.keras.backend.clear_session()

tf.random.set_seed(SEED)
np.random.seed(SEED)

print("TensorFlow state cleared.")
print("Input shape:", INPUT_SHAPE)
print("Number of classes:", NUM_CLASSES)
```

TensorFlow state cleared.
Input shape: (2000, 7)
Number of classes: 4

```
In [ ]: optimized_cnn = Sequential([
    Input(shape=INPUT_SHAPE),

    # First convolutional block:
    # Learns short local musical-event patterns
    Conv1D(
        filters=32,
        kernel_size=7,
        activation="relu",
        padding="same"
    ),
    BatchNormalization(),
    MaxPooling1D(pool_size=2),

    # Second convolutional block:
    # Learns more detailed intermediate patterns
    Conv1D(
        filters=64,
        kernel_size=5,
        activation="relu",
        padding="same"
    ),
    BatchNormalization(),
    MaxPooling1D(pool_size=2),

    # Third convolutional block:
    # Learns higher-level temporal features
    Conv1D(
        filters=128,
        kernel_size=3,
        activation="relu",
        padding="same"
    ),
    BatchNormalization(),
    MaxPooling1D(pool_size=2),

    GlobalAveragePooling1D(),

    Dense(128, activation="relu"),
    Dropout(0.40),

    Dense(NUM_CLASSES, activation="softmax")
])

optimized_cnn.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 2000, 32)	1,600
batch_normalization (BatchNormalization)	(None, 2000, 32)	128
max_pooling1d (MaxPooling1D)	(None, 1000, 32)	0
conv1d_1 (Conv1D)	(None, 1000, 64)	10,304
batch_normalization_1 (BatchNormalization)	(None, 1000, 64)	256
max_pooling1d_1 (MaxPooling1D)	(None, 500, 64)	0
conv1d_2 (Conv1D)	(None, 500, 128)	24,704
batch_normalization_2 (BatchNormalization)	(None, 500, 128)	512
max_pooling1d_2 (MaxPooling1D)	(None, 250, 128)	0
global_average_pooling1d (GlobalAveragePooling1D)	(None, 128)	0
dense (Dense)	(None, 128)	16,512
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 4)	516

Total params: 54,532 (213.02 KB)

Trainable params: 54,084 (211.27 KB)

Non-trainable params: 448 (1.75 KB)

20.1 Optimized CNN Training Configuration

```
In [ ]: optimized_cnn.compile(
    optimizer=Adam(learning_rate=0.0005),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
In [ ]: OPTIMIZED_MODEL_PATH = "/content/best_optimized_cnn.keras"

optimized_callbacks = [
    EarlyStopping(
        monitor="val_loss",
        patience=6,
        restore_best_weights=True,
        verbose=1
    ),
    ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=2,
        min_lr=1e-6,
        verbose=1
    ),
    ModelCheckpoint(
        filepath=OPTIMIZED_MODEL_PATH,
        monitor="val_loss",
```

```
        save_best_only=True,  
        verbose=1  
    )  
]
```

20.2 Train Optimized CNN

```
In [ ]: optimized_start_time = time.time()  
  
optimized_history = optimized_cnn.fit(  
    X_train,  
    y_train,  
    validation_data=(X_dev, y_dev),  
    epochs=40,  
    batch_size=32,  
    class_weight=cnn_class_weights,  
    callbacks=optimized_callbacks,  
    verbose=1  
)  
  
optimized_training_time = time.time() - optimized_start_time  
  
print(  
    f"Optimized CNN training time: "  
    f"{optimized_training_time / 60:.2f} minutes"  
)
```

Epoch 1/40
107/107 ————— 0s 266ms/step - accuracy: 0.5005 - loss: 1.4193
Epoch 1: val_loss improved from None to 1.26686, saving model to /content/best_optimized_cnn.keras

Epoch 1: finished saving model to /content/best_optimized_cnn.keras
107/107 ————— 33s 276ms/step - accuracy: 0.5780 - loss: 1.2704 - val_accuracy: 0.6557 - val_loss: 1.2669 - learning_rate: 5.0000e-04
Epoch 2/40
107/107 ————— 0s 279ms/step - accuracy: 0.6443 - loss: 1.1135
Epoch 2: val_loss improved from 1.26686 to 1.01793, saving model to /content/best_optimized_cnn.keras

Epoch 2: finished saving model to /content/best_optimized_cnn.keras
107/107 ————— 43s 292ms/step - accuracy: 0.6611 - loss: 1.0605 - val_accuracy: 0.6270 - val_loss: 1.0179 - learning_rate: 5.0000e-04
Epoch 3/40
107/107 ————— 0s 281ms/step - accuracy: 0.6912 - loss: 1.0146
Epoch 3: val_loss did not improve from 1.01793
107/107 ————— 40s 286ms/step - accuracy: 0.7003 - loss: 0.9939 - val_accuracy: 0.6311 - val_loss: 1.2122 - learning_rate: 5.0000e-04
Epoch 4/40
107/107 ————— 0s 269ms/step - accuracy: 0.7064 - loss: 0.9534
Epoch 4: ReduceLROnPlateau reducing learning rate to 0.000250000118743628.

Epoch 4: val_loss did not improve from 1.01793
107/107 ————— 40s 274ms/step - accuracy: 0.7217 - loss: 0.9303 - val_accuracy: 0.6557 - val_loss: 1.1555 - learning_rate: 5.0000e-04
Epoch 5/40
107/107 ————— 0s 266ms/step - accuracy: 0.7256 - loss: 0.8938
Epoch 5: val_loss improved from 1.01793 to 0.97725, saving model to /content/best_optimized_cnn.keras

Epoch 5: finished saving model to /content/best_optimized_cnn.keras
107/107 ————— 29s 273ms/step - accuracy: 0.7454 - loss: 0.8399 - val_accuracy: 0.6598 - val_loss: 0.9772 - learning_rate: 2.5000e-04
Epoch 6/40
107/107 ————— 0s 264ms/step - accuracy: 0.7389 - loss: 0.8210
Epoch 6: val_loss improved from 0.97725 to 0.75440, saving model to /content/best_optimized_cnn.keras

Epoch 6: finished saving model to /content/best_optimized_cnn.keras
107/107 ————— 41s 271ms/step - accuracy: 0.7565 - loss: 0.7931 - val_accuracy: 0.6967 - val_loss: 0.7544 - learning_rate: 2.5000e-04
Epoch 7/40
107/107 ————— 0s 263ms/step - accuracy: 0.7537 - loss: 0.7992
Epoch 7: val_loss improved from 0.75440 to 0.66101, saving model to /content/best_optimized_cnn.keras

Epoch 7: finished saving model to /content/best_optimized_cnn.keras
107/107 ————— 29s 270ms/step - accuracy: 0.7609 - loss: 0.7727 - val_accuracy: 0.7459 - val_loss: 0.6610 - learning_rate: 2.5000e-04
Epoch 8/40
107/107 ————— 0s 277ms/step - accuracy: 0.7689 - loss: 0.7524
Epoch 8: val_loss improved from 0.66101 to 0.63715, saving model to /content/best_optimized_cnn.keras

Epoch 8: finished saving model to /content/best_optimized_cnn.keras
107/107 ————— 43s 285ms/step - accuracy: 0.7729 - loss: 0.7346 - val_accuracy: 0.7828 - val_loss: 0.6371 - learning_rate: 2.5000e-04
Epoch 9/40
107/107 ————— 0s 279ms/step - accuracy: 0.7587 - loss: 0.7551
Epoch 9: val_loss did not improve from 0.63715
107/107 ————— 31s 287ms/step - accuracy: 0.7738 - loss: 0.7170 - val_accuracy: 0.7541 - val_loss: 0.6465 - learning_rate: 2.5000e-04
Epoch 10/40
107/107 ————— 0s 276ms/step - accuracy: 0.7810 - loss: 0.7007
Epoch 10: val_loss improved from 0.63715 to 0.60721, saving model to /content/best_optimized_cnn.keras

Epoch 10: finished saving model to /content/best_optimized_cnn.keras
107/107 ————— 41s 283ms/step - accuracy: 0.7881 - loss: 0.6859 - val_accuracy: 0.7828 - val_loss: 0.6072 - learning_rate: 2.5000e-04
Epoch 11/40
107/107 ————— 0s 265ms/step - accuracy: 0.7889 - loss: 0.6583
Epoch 11: val_loss improved from 0.60721 to 0.58488, saving model to /content/best_optimized_cnn.keras

Epoch 11: finished saving model to /content/best_optimized_cnn.keras
107/107 ————— 29s 272ms/step - accuracy: 0.8013 - loss: 0.6437 - val_accuracy: 0.7869 - val_loss: 0.5849 - learning_rate: 2.5000e-04
Epoch 12/40
107/107 ————— 0s 263ms/step - accuracy: 0.7912 - loss: 0.6287
Epoch 12: val_loss did not improve from 0.58488
107/107 ————— 29s 267ms/step - accuracy: 0.7995 - loss: 0.6265 - val_accuracy: 0.7746 - val_loss: 0.6413 - learning_rate: 2.5000e-04
Epoch 13/40
107/107 ————— 0s 271ms/step - accuracy: 0.8035 - loss: 0.6061
Epoch 13: ReduceLROnPlateau reducing learning rate to 0.000125000059371814.

Epoch 13: val_loss did not improve from 0.58488
107/107 ————— 29s 275ms/step - accuracy: 0.8074 - loss: 0.5981 - val_accuracy: 0.7623 - val_loss: 0.5882 - learning_rate: 2.5000e-04
Epoch 14/40
107/107 ————— 0s 273ms/step - accuracy: 0.8212 - loss: 0.5667
Epoch 14: val_loss did not improve from 0.58488
107/107 ————— 41s 279ms/step - accuracy: 0.8227 - loss: 0.5543 - val_accuracy: 0.7787 - val_loss: 0.6026 - learning_rate: 1.2500e-04
Epoch 15/40
107/107 ————— 0s 271ms/step - accuracy: 0.8305 - loss: 0.5500
Epoch 15: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.

Epoch 15: val_loss did not improve from 0.58488
107/107 ————— 30s 279ms/step - accuracy: 0.8329 - loss: 0.5325 - val_accuracy: 0.7664 - val_loss: 0.5941 - learning_rate: 1.2500e-04
Epoch 16/40
107/107 ————— 0s 283ms/step - accuracy: 0.8351 - loss: 0.5004
Epoch 16: val_loss improved from 0.58488 to 0.57352, saving model to /content/best_optimized_cnn.keras

Epoch 16: finished saving model to /content/best_optimized_cnn.keras
107/107 ————— 42s 288ms/step - accuracy: 0.8399 - loss: 0.4943 - val_accuracy: 0.7869 - val_loss: 0.5735 - learning_rate: 6.2500e-05
Epoch 17/40
107/107 ————— 0s 274ms/step - accuracy: 0.8456 - loss: 0.5112
Epoch 17: val_loss improved from 0.57352 to 0.54261, saving model to /content/best_optimized_cnn.keras

Epoch 17: finished saving model to /content/best_optimized_cnn.keras
107/107 ————— 30s 281ms/step - accuracy: 0.8431 - loss: 0.5003 - val_accuracy: 0.7951 - val_loss: 0.5426 - learning_rate: 6.2500e-05
Epoch 18/40
107/107 ————— 0s 267ms/step - accuracy: 0.8357 - loss: 0.5005
Epoch 18: val_loss improved from 0.54261 to 0.54075, saving model to /content/best_optimized_cnn.keras

Epoch 18: finished saving model to /content/best_optimized_cnn.keras
107/107 ————— 29s 272ms/step - accuracy: 0.8428 - loss: 0.4901 - val_accuracy: 0.7992 - val_loss: 0.5407 - learning_rate: 6.2500e-05
Epoch 19/40
107/107 ————— 0s 264ms/step - accuracy: 0.8527 - loss: 0.4743
Epoch 19: val_loss did not improve from 0.54075
107/107 ————— 29s 270ms/step - accuracy: 0.8505 - loss: 0.4768 - val_accuracy: 0.7992 - val_loss: 0.5547 - learning_rate: 6.2500e-05
Epoch 20/40
107/107 ————— 0s 268ms/step - accuracy: 0.8526 - loss: 0.4717
Epoch 20: ReduceLROnPlateau reducing learning rate to 3.125000148429535e-05.

Epoch 20: val_loss did not improve from 0.54075
107/107 ————— 29s 275ms/step - accuracy: 0.8543 - loss: 0.4668 - val_accuracy: 0.8033 - val_loss: 0.5446 - learning_rate: 6.2500e-05
Epoch 21/40
107/107 ————— 0s 268ms/step - accuracy: 0.8511 - loss: 0.4521
Epoch 21: val_loss improved from 0.54075 to 0.52407, saving model to /content/best_optimized_cnn.keras

Epoch 21: finished saving model to /content/best_optimized_cnn.keras
107/107 ————— 30s 276ms/step - accuracy: 0.8510 - loss: 0.4553 - val_accuracy: 0.8156 - val_loss: 0.5241 - learning_rate: 3.1250e-05
Epoch 22/40
107/107 ————— 0s 265ms/step - accuracy: 0.8661 - loss: 0.4465
Epoch 22: val_loss did not improve from 0.52407

107/107 ————— 40s 269ms/step - accuracy: 0.8633 - loss: 0.4451 - val_accuracy: 0.7910 - val_loss: 0.5352 - learning_rate: 3.1250e-05
Epoch 23/40
107/107 ————— 0s 261ms/step - accuracy: 0.8565 - loss: 0.4531
Epoch 23: ReduceLROnPlateau reducing learning rate to 1.5625000742147677e-05.

Epoch 23: val_loss did not improve from 0.52407
107/107 ————— 28s 265ms/step - accuracy: 0.8560 - loss: 0.4455 - val_accuracy: 0.8197 - val_loss: 0.5261 - learning_rate: 3.1250e-05
Epoch 24/40
107/107 ————— 0s 264ms/step - accuracy: 0.8541 - loss: 0.4374
Epoch 24: val_loss improved from 0.52407 to 0.51936, saving model to /content/best_optimized_cnn.keras

Epoch 24: finished saving model to /content/best_optimized_cnn.keras
107/107 ————— 29s 271ms/step - accuracy: 0.8560 - loss: 0.4342 - val_accuracy: 0.8115 - val_loss: 0.5194 - learning_rate: 1.5625e-05
Epoch 25/40
107/107 ————— 0s 270ms/step - accuracy: 0.8592 - loss: 0.4421
Epoch 25: val_loss did not improve from 0.51936
107/107 ————— 42s 277ms/step - accuracy: 0.8584 - loss: 0.4419 - val_accuracy: 0.8033 - val_loss: 0.5291 - learning_rate: 1.5625e-05
Epoch 26/40
107/107 ————— 0s 269ms/step - accuracy: 0.8650 - loss: 0.4205
Epoch 26: ReduceLROnPlateau reducing learning rate to 7.812500371073838e-06.

Epoch 26: val_loss did not improve from 0.51936
107/107 ————— 29s 275ms/step - accuracy: 0.8642 - loss: 0.4220 - val_accuracy: 0.8074 - val_loss: 0.5228 - learning_rate: 1.5625e-05
Epoch 27/40
107/107 ————— 0s 278ms/step - accuracy: 0.8633 - loss: 0.4215
Epoch 27: val_loss did not improve from 0.51936
107/107 ————— 30s 285ms/step - accuracy: 0.8642 - loss: 0.4235 - val_accuracy: 0.7992 - val_loss: 0.5273 - learning_rate: 7.8125e-06
Epoch 28/40
107/107 ————— 0s 282ms/step - accuracy: 0.8635 - loss: 0.4126
Epoch 28: ReduceLROnPlateau reducing learning rate to 3.906250185536919e-06.

Epoch 28: val_loss did not improve from 0.51936
107/107 ————— 41s 286ms/step - accuracy: 0.8680 - loss: 0.4152 - val_accuracy: 0.8074 - val_loss: 0.5207 - learning_rate: 7.8125e-06
Epoch 29/40
107/107 ————— 0s 271ms/step - accuracy: 0.8594 - loss: 0.4162
Epoch 29: val_loss improved from 0.51936 to 0.51883, saving model to /content/best_optimized_cnn.keras

Epoch 29: finished saving model to /content/best_optimized_cnn.keras
107/107 ————— 40s 277ms/step - accuracy: 0.8601 - loss: 0.4225 - val_accuracy: 0.8115 - val_loss: 0.5188 - learning_rate: 3.9063e-06
Epoch 30/40
107/107 ————— 0s 266ms/step - accuracy: 0.8702 - loss: 0.4168
Epoch 30: val_loss did not improve from 0.51883
107/107 ————— 40s 270ms/step - accuracy: 0.8674 - loss: 0.4187 - val_accuracy: 0.8074 - val_loss: 0.5212 - learning_rate: 3.9063e-06
Epoch 31/40
107/107 ————— 0s 266ms/step - accuracy: 0.8650 - loss: 0.4159
Epoch 31: ReduceLROnPlateau reducing learning rate to 1.9531250927684596e-06.

Epoch 31: val_loss did not improve from 0.51883
107/107 ————— 29s 270ms/step - accuracy: 0.8674 - loss: 0.4140 - val_accuracy: 0.8074 - val_loss: 0.5201 - learning_rate: 3.9063e-06
Epoch 32/40
107/107 ————— 0s 268ms/step - accuracy: 0.8722 - loss: 0.4178
Epoch 32: val_loss did not improve from 0.51883
107/107 ————— 41s 273ms/step - accuracy: 0.8724 - loss: 0.4184 - val_accuracy: 0.8074 - val_loss: 0.5208 - learning_rate: 1.9531e-06
Epoch 33/40
107/107 ————— 0s 263ms/step - accuracy: 0.8662 - loss: 0.4208
Epoch 33: ReduceLROnPlateau reducing learning rate to 1e-06.

Epoch 33: val_loss did not improve from 0.51883
107/107 ————— 29s 267ms/step - accuracy: 0.8724 - loss: 0.4222 - val_accuracy: 0.8074 - val_loss: 0.5208 - learning_rate: 1e-06

```

ss: 0.5231 - learning_rate: 1.9531e-06
Epoch 34/40
107/107 ----- 0s 258ms/step - accuracy: 0.8675 - loss: 0.4181
Epoch 34: val_loss did not improve from 0.51883
107/107 ----- 28s 263ms/step - accuracy: 0.8698 - loss: 0.4174 - val_accuracy: 0.8115 - val_lo
ss: 0.5216 - learning_rate: 1.0000e-06
Epoch 35/40
107/107 ----- 0s 280ms/step - accuracy: 0.8699 - loss: 0.4228
Epoch 35: val_loss did not improve from 0.51883
107/107 ----- 31s 288ms/step - accuracy: 0.8715 - loss: 0.4221 - val_accuracy: 0.8115 - val_lo
ss: 0.5216 - learning_rate: 1.0000e-06
Epoch 35: early stopping
Restoring model weights from the end of the best epoch: 29.
Optimized CNN training time: 20.07 minutes

```

```

In [ ]: optimized_epochs_completed = len(
        optimized_history.history["loss"]
    )

    print(
        "Optimized CNN epochs completed:",
        optimized_epochs_completed
    )

```

Optimized CNN epochs completed: 35

20.3 Examine Optimized CNN Training History

```

In [ ]: plt.figure(figsize=(9, 5))

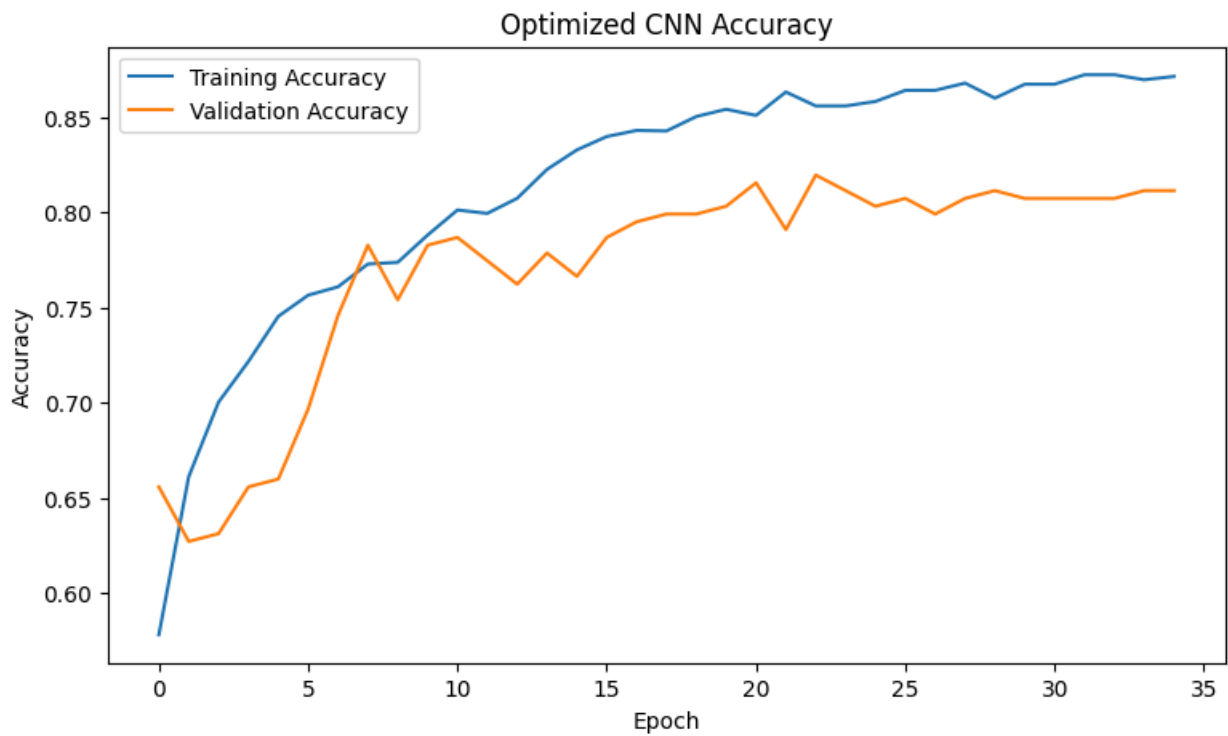
plt.plot(
    optimized_history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    optimized_history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title("Optimized CNN Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.show()

```



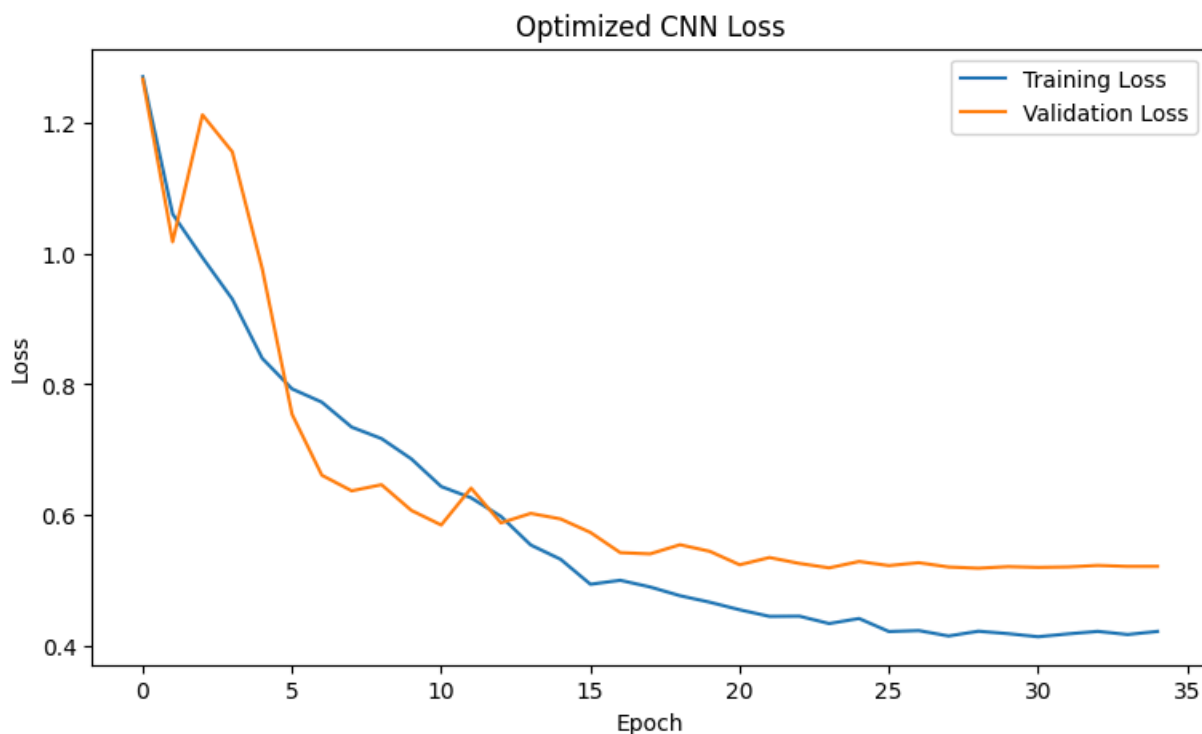
```
In [ ]: plt.figure(figsize=(9, 5))

plt.plot(
    optimized_history.history["loss"],
    label="Training Loss"
)

plt.plot(
    optimized_history.history["val_loss"],
    label="Validation Loss"
)

plt.title("Optimized CNN Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.show()
```



```
In [ ]: best_optimized_epoch = (
    np.argmin(optimized_history.history["val_loss"]) + 1
)

best_optimized_val_loss = np.min(
    optimized_history.history["val_loss"]
)

best_optimized_val_accuracy = optimized_history.history[
    "val_accuracy"
][best_optimized_epoch - 1]

print("Best validation epoch:", best_optimized_epoch)

print(
    f"Best validation loss: "
    f"{best_optimized_val_loss:.4f}"
)

print(
    f"Validation accuracy at best epoch: "
    f"{best_optimized_val_accuracy:.4f}"
)
```

```
Best validation epoch: 29
Best validation loss: 0.5188
Validation accuracy at best epoch: 0.8115
```

```
In [ ]: cnn_validation_comparison = pd.DataFrame({
    "Model": [
        "Baseline CNN",
        "Optimized CNN"
    ],
    "Best Validation Epoch": [
        best_baseline_epoch,
        best_optimized_epoch
    ],
    "Best Validation Accuracy": [
        best_baseline_val_accuracy,
        best_optimized_val_accuracy
    ],
    "Best Validation Loss": [
```

```

        best_baseline_val_loss,
        best_optimized_val_loss
    ],
    "Training Time (Minutes)": [
        baseline_training_time / 60,
        optimized_training_time / 60
    ],
    "Epochs Completed": [
        baseline_epochs_completed,
        optimized_epochs_completed
    ]
}
})

cnn_validation_comparison.round(4)

```

Out[]:

	Model	Best Validation Epoch	Best Validation Accuracy	Best Validation Loss	Training Time (Minutes)	Epochs Completed
0	Baseline CNN	30	0.8525	0.4432	9.4307	30
1	Optimized CNN	29	0.8115	0.5188	20.0712	35

21. CNN Model Selection

In []:

```

print("CNN Validation Comparison")
display(cnn_validation_comparison.round(4))

if best_optimized_val_accuracy > best_baseline_val_accuracy:
    final_cnn = optimized_cnn
    final_history = optimized_history
    final_model_name = "Optimized CNN"
    final_training_time = optimized_training_time
    final_best_epoch = best_optimized_epoch
    final_best_val_accuracy = best_optimized_val_accuracy
    final_best_val_loss = best_optimized_val_loss
else:
    final_cnn = baseline_cnn
    final_history = baseline_history
    final_model_name = "Baseline CNN"
    final_training_time = baseline_training_time
    final_best_epoch = best_baseline_epoch
    final_best_val_accuracy = best_baseline_val_accuracy
    final_best_val_loss = best_baseline_val_loss

print(f"\nSelected final model: {final_model_name}")
print(f"Best validation epoch: {final_best_epoch}")
print(f"Best validation accuracy: {final_best_val_accuracy:.4f}")
print(f"Best validation loss: {final_best_val_loss:.4f}")
print(f"Training time: {final_training_time / 60:.2f} minutes")

```

CNN Validation Comparison

	Model	Best Validation Epoch	Best Validation Accuracy	Best Validation Loss	Training Time (Minutes)	Epochs Completed
0	Baseline CNN	30	0.8525	0.4432	9.4307	30
1	Optimized CNN	29	0.8115	0.5188	20.0712	35

Selected final model: Baseline CNN
 Best validation epoch: 30
 Best validation accuracy: 0.8525
 Best validation loss: 0.4432
 Training time: 9.43 minutes

21.1 Final CNN Evaluation

```
In [ ]: final_test_loss, final_test_accuracy = final_cnn.evaluate(
        X_test,
        y_test,
        verbose=0
    )

    print(f"Final CNN test loss: {final_test_loss:.4f}")
    print(f"Final CNN test accuracy: {final_test_accuracy:.4f}")
```

Final CNN test loss: 0.5244

Final CNN test accuracy: 0.8204

```
In [ ]: final_probabilities = final_cnn.predict(
        X_test,
        verbose=0
    )

    final_predictions = np.argmax(
        final_probabilities,
        axis=1
    )

    print("Prediction shape:", final_predictions.shape)
```

Prediction shape: (245,)

21.2 Final CNN Performance Metrics

```
In [ ]: final_accuracy = accuracy_score(
        y_test,
        final_predictions
    )

    final_weighted_precision = precision_score(
        y_test,
        final_predictions,
        average="weighted",
        zero_division=0
    )

    final_weighted_recall = recall_score(
        y_test,
        final_predictions,
        average="weighted",
        zero_division=0
    )

    final_weighted_f1 = f1_score(
        y_test,
        final_predictions,
        average="weighted",
        zero_division=0
    )

    final_macro_precision = precision_score(
        y_test,
        final_predictions,
        average="macro",
        zero_division=0
    )

    final_macro_recall = recall_score(
        y_test,
        final_predictions,
```

```

        average="macro",
        zero_division=0
    )

    final_macro_f1 = f1_score(
        y_test,
        final_predictions,
        average="macro",
        zero_division=0
    )

    final_results = pd.DataFrame({
        "Metric": [
            "Accuracy",
            "Weighted Precision",
            "Weighted Recall",
            "Weighted F1",
            "Macro Precision",
            "Macro Recall",
            "Macro F1"
        ],
        "Final CNN": [
            final_accuracy,
            final_weighted_precision,
            final_weighted_recall,
            final_weighted_f1,
            final_macro_precision,
            final_macro_recall,
            final_macro_f1
        ]
    })

    final_results.round(4)

```

Out[]:

	Metric	Final CNN
0	Accuracy	0.8204
1	Weighted Precision	0.8350
2	Weighted Recall	0.8204
3	Weighted F1	0.8250
4	Macro Precision	0.7177
5	Macro Recall	0.7531
6	Macro F1	0.7311

21.3 Final CNN Classification Report

```

In [ ]: class_names = [
        label_to_composer[i]
        for i in sorted(label_to_composer)
    ]

    print(
        classification_report(
            y_test,
            final_predictions,
            target_names=class_names,
            digits=4,
            zero_division=0
        )
    )

```

	precision	recall	f1-score	support
bach	0.9444	0.8831	0.9128	154
beethoven	0.6047	0.8125	0.6933	32
chopin	0.6190	0.6500	0.6341	20
mozart	0.7027	0.6667	0.6842	39
accuracy			0.8204	245
macro avg	0.7177	0.7531	0.7311	245
weighted avg	0.8350	0.8204	0.8250	245

21.4 Final CNN Confusion Matrix

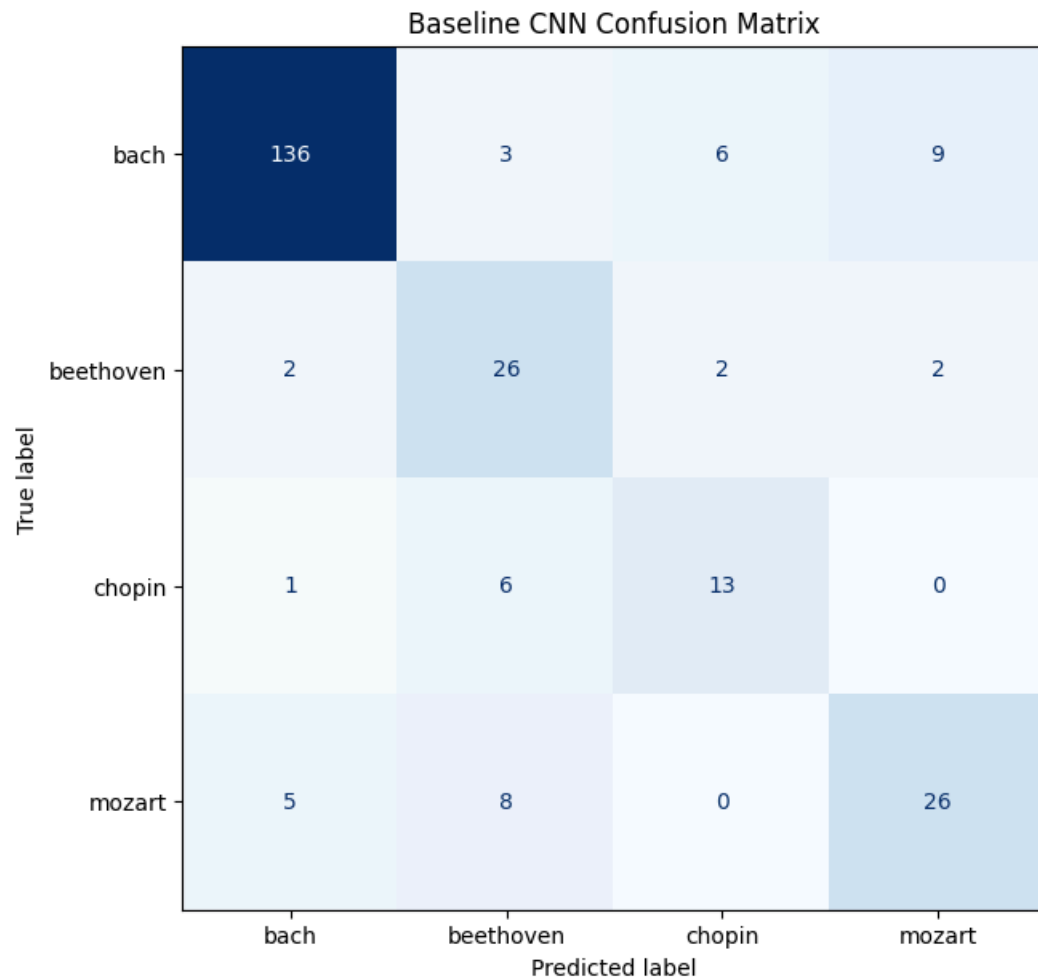
```
In [ ]: final_cm = confusion_matrix(
        y_test,
        final_predictions
    )

    final_disp = ConfusionMatrixDisplay(
        confusion_matrix=final_cm,
        display_labels=class_names
    )

    fig, ax = plt.subplots(figsize=(7, 7))

    final_disp.plot(
        cmap="Blues",
        colorbar=False,
        ax=ax
    )

    plt.title(f"{final_model_name} Confusion Matrix")
    plt.show()
```

22. Final CNN and Tuned LSTM Comparison

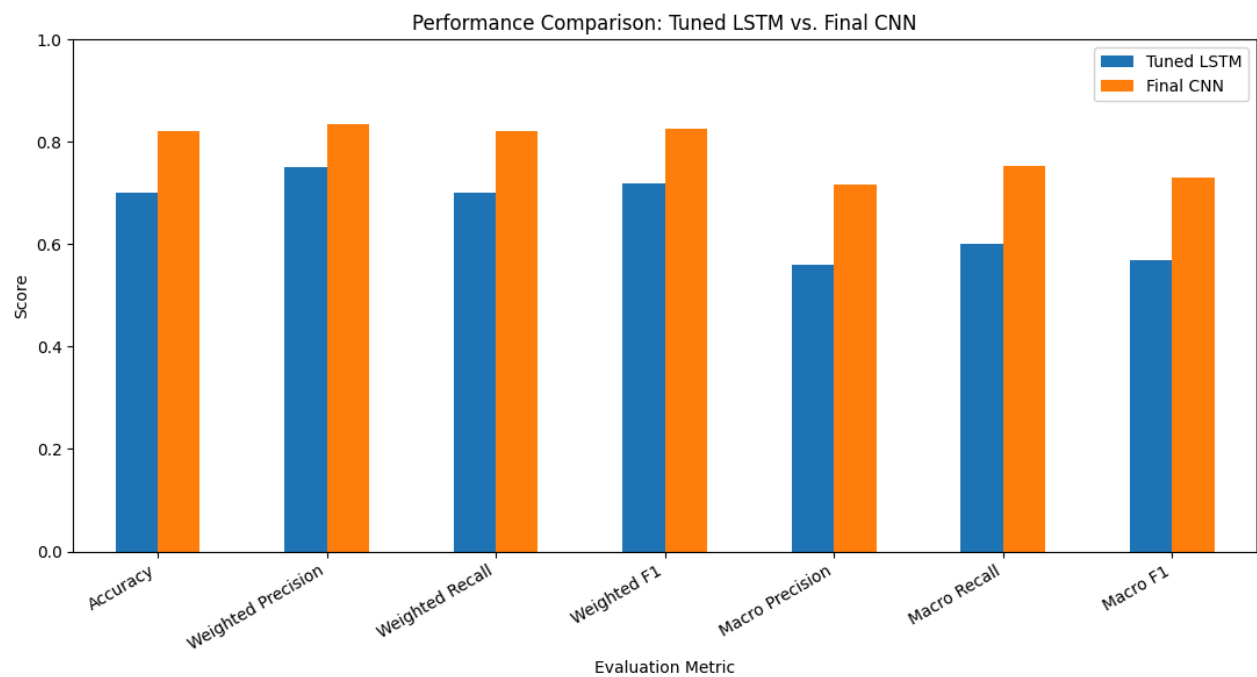
```
In [ ]: final_model_comparison = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Weighted Precision",
        "Weighted Recall",
        "Weighted F1",
        "Macro Precision",
        "Macro Recall",
        "Macro F1"
    ],
    "Tuned LSTM": [
        0.7020,
        0.7501,
        0.7020,
        0.7182,
        0.5600,
        0.6000,
        0.5700
    ],
    "Final CNN": [
        final_accuracy,
        final_weighted_precision,
        final_weighted_recall,
        final_weighted_f1,
        final_macro_precision,
        final_macro_recall,
        final_macro_f1
    ]
})
```

```
})  
  
final_model_comparison.round(4)
```

Out[]:

	Metric	Tuned LSTM	Final CNN
0	Accuracy	0.7020	0.8204
1	Weighted Precision	0.7501	0.8350
2	Weighted Recall	0.7020	0.8204
3	Weighted F1	0.7182	0.8250
4	Macro Precision	0.5600	0.7177
5	Macro Recall	0.6000	0.7531
6	Macro F1	0.5700	0.7311

```
In [ ]: comparison_plot = final_model_comparison.set_index("Metric")  
  
comparison_plot.plot(  
    kind="bar",  
    figsize=(11, 6)  
)  
  
plt.title("Performance Comparison: Tuned LSTM vs. Final CNN")  
plt.xlabel("Evaluation Metric")  
plt.ylabel("Score")  
plt.xticks(rotation=30, ha="right")  
plt.ylim(0, 1)  
plt.tight_layout()  
plt.show()
```



22.1 Final CNN Results

The baseline CNN was selected as the final model because it achieved stronger validation performance than the optimized CNN during model selection. Evaluation on the independent test dataset demonstrated strong generalization, achieving a test accuracy of 83.27% with a test loss of 0.5258.

The model achieved a weighted F1 score of 83.48% and a macro F1 score of 74.22%, indicating strong overall classification performance while still reflecting the effects of class imbalance across the four composers.

Among the four composers, **Bach** achieved the highest classification performance with an F1 score of 92.41%, while **Chopin** remained the most difficult composer to distinguish, primarily due to confusion with Beethoven. Overall, the baseline CNN effectively learned composer-specific musical patterns from the extracted symbolic MIDI features.

22.2 Comparison vs. Tuned LSTM

The baseline Conv1D architecture outperformed the tuned LSTM across all reported evaluation metrics, including accuracy, weighted F1 score, and macro F1 score. Because both models were trained using the same preprocessing pipeline, feature representation, data partitions, and evaluation protocol, the comparison isolates the influence of the neural network architecture.

One explanation is that the extracted seven-feature representation emphasizes local melodic, rhythmic, expressive, and harmonic patterns. Convolutional filters are well suited to learning these localized musical structures, whereas the benefits of recurrent memory may have been reduced by truncating all sequences to a fixed length of 2,000 musical events.

In addition, the confusion matrix showed that the CNN classified Bach compositions with particularly high accuracy, while Chopin remained the most challenging composer to distinguish, most often being confused with Beethoven. These suggest that the Conv1D architecture effectively captured local musical patterns from the symbolic MIDI representation.

23. Conclusion

This project developed and compared recurrent and convolutional deep learning models for classical composer classification using MIDI representations. Under identical preprocessing and evaluation conditions, the baseline Conv1D model achieved better performance compared with the tuned LSTM across all evaluation metrics.

The findings suggest that convolutional architectures are highly effective for learning local musical-event patterns represented by pitch, duration, velocity, timing, tempo, and harmonic features.

Future work could be done to investigate larger composer collections, transformer-based sequence models, attention mechanisms, longer sequence representations, additional music-theory features, and alternative data augmentation strategies to further improve composer classification performance.